Chapter 6. I/O system

The Windows I/O system consists of several executive components that, together, manage hardware devices and provide interfaces to hardware devices for applications and the system. This chapter lists the design goals of the I/O system, which have influenced its implementation. It then covers the components that make up the I/O system, including the I/O manager, Plug and Play (PnP) manager, and power manager. Then it examines the structure and components of the I/O system and the various types of device drivers. It discusses the key data structures that describe devices, device drivers, and I/O requests, after which it describes the steps necessary to complete I/O requests as they move through the system. Finally, it presents the way device detection, driver installation, and power management work.

I/O system components

The design goals for the Windows I/O system are to provide an abstraction of devices, both hardware (physical) and software (virtual or logical), to applications with the following features:

- Uniform security and naming across devices to protect shareable resources. (See **Chapter 7**, "**Security**," for a description of the Windows security model.)

- High-performance asynchronous packet-based I/O to allow for the implementation of scalable applications.

- Services that allow drivers to be written in a high-level language and easily ported between different machine architectures.

■ Layering and extensibility to allow for the addition of drivers that transparently modify the behavior of other drivers or devices, without requiring any changes to the driver whose behavior or device is modified.

■ Dynamic loading and unloading of device drivers so that drivers can be loaded on demand and not consume system resources when unneeded.

■ Support for Plug and Play, where the system locates and installs drivers for newly detected hardware, assigns them hardware resources they require, and allows applications to discover and activate device interfaces.

■ Support for power management so that the system or individual devices can enter low-power states.

■ Support for multiple installable file systems, including FAT (and its variants, FAT32 and exFAT), the CD-ROM file system (CDFS), the Universal Disk Format (UDF) file system, the Resilient File System (ReFS), and the Windows file system (NTFS). (See Chapter 13, "File systems," in Part 2 of this book for more specific information on file system types and architecture.)

■ Windows Management Instrumentation (WMI) support and diagnosability so that drivers can be managed and monitored through WMI applications and scripts. (WMI is described in Chapter 9, "Management mechanisms," in Part 2.)

To implement these features, the Windows I/O system consists of several executive components as well as device drivers, which are shown in **Figure 6-1**.

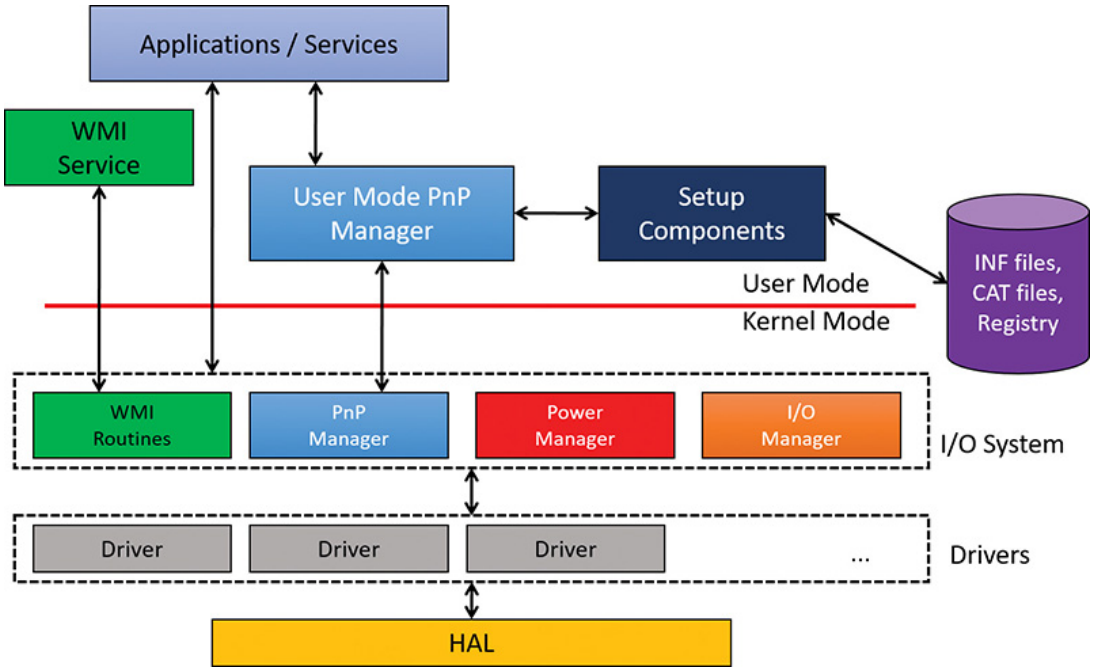


FIGURE 6-1 I/O system components.

■ The I/O manager is the heart of the I/O system. It connects applications and system components to virtual, logical, and physical devices, and it defines the infrastructure that supports device drivers.

■ A device driver typically provides an I/O interface for a particular type of device. A *driver* is a software module that interprets high-level commands, such as read or write commands, and issues low-level, device-specific commands, such as writing to control registers. Device drivers receive commands routed to them by the I/O manager that are directed at the devices they manage, and they inform the I/O manager when those commands are complete. Device drivers often use the I/O manager to forward I/O commands to other device drivers that share in the implementation of a device's interface or control.

■ The PnP manager works closely with the I/O manager and a type of device driver called *a bus driver* to guide the allocation of hardware resources as well as to detect and respond to the arrival and removal of hardware devices. The PnP manager and bus drivers are responsible for loading a device's driver when the device is detected. When a device is added to a system that doesn't have an appropriate device driver, the executive Plug and Play component calls on the device-installation services of the user-mode PnP manager.

■ The power manager also works closely with the I/O manager and the PnP manager to guide the system, as well as individual device drivers, through power-state transitions.

■ WMI support routines, called the Windows Driver Model (WDM) WMI provider, allow device drivers to indirectly act as providers, using the WDM WMI provider as an intermediary to communicate with the WMI service in user mode.

■ The registry serves as a database that stores a description of basic hardware devices attached to the system as well as driver initialization and configuration settings. (See the section “The registry” in Chapter 9 in Part 2 for more information.)

■ INF files, which are designated by the .inf extension, are driver-installation files. INF files are the link between a particular hardware device and the driver that assumes primary control of that device. They are made up of script-like instructions describing the device they correspond to, the source and target locations of driver files, required driver-installation registry modifications, and driver-dependency information. Digital signatures that Windows uses to verify that a driver file has passed testing by the Microsoft Windows Hardware Quality Labs (WHQL) are stored in .cat files. Digital signatures are also used to prevent tampering of the driver or its INF file.

■ The hardware abstraction layer (HAL) insulates drivers from the specifics of the processor and interrupt controller by providing APIs that hide differences between platforms. In essence, the HAL is the bus driver for all the devices soldered onto the computer’s motherboard that aren’t controlled by other drivers.

The I/O manager

The I/O manager is the core of the I/O system. It defines the orderly framework, or model, within which I/O requests are delivered to device drivers. The I/O system is packet driven. Most I/O requests are represented by an I/O request packet (IRP), which is a data structure that contains information completely describing an I/O request. The IRP travels from one I/O system component to another. (As you'll discover in the section "**Fast I/O**," fast I/O is the exception; it doesn't use IRPs.) The design allows an individual application thread to manage multiple I/O requests concurrently. (For more information on IRPs, see the section "**I/O request packets**" later in this chapter.)

The I/O manager creates an IRP in memory to represent an I/O operation, passing a pointer to the IRP to the correct driver and disposing of the packet when the I/O operation is complete. In contrast, a driver receives an IRP, performs the operation the IRP specifies, and passes the IRP back to the I/O manager, either because the requested I/O operation has been completed or because it must be passed on to another driver for further processing.

In addition to creating and disposing of IRPs, the I/O manager supplies code that is common to different drivers and that the drivers can call to carry out their I/O processing. By consolidating common tasks in the I/O manager, individual drivers become simpler and more compact. For example, the I/O manager provides a function that allows one driver to call other drivers. It also manages buffers for I/O requests, provides timeout support for drivers, and records which installable file systems are loaded into the operating system. There are about 100 different routines in the I/O manager that can be called by device drivers.

The I/O manager also provides flexible I/O services that allow environment subsystems, such as Windows and POSIX (the latter is no longer supported), to implement their respective I/O functions. These services include support for asynchronous I/O that allow developers to build scalable, high-performance server applications.

The uniform, modular interface that drivers present allows the I/O manager to call any driver without requiring any special knowledge of its structure or internal details. The operating system treats all I/O requests as if they were directed at a file; the driver converts the requests from requests made to a virtual file to hardware-specific requests. Drivers can also call each other (using the I/O manager) to achieve layered, independent processing of an I/O request.

Besides providing the normal open, close, read, and write functions, the Windows I/O system provides several advanced features, such as asynchronous, direct, buffered, and scatter/gather I/O, which are described in the "**Types of I/O**" section later in this chapter.

Typical I/O processing

Most I/O operations don't involve all the components of the I/O system. A typical I/O request starts with an application executing an I/O-related function (for example, reading data from a device) that is processed by the I/O manager, one or more device drivers, and the HAL.

As mentioned, in Windows, threads perform I/O on virtual files. A virtual file refers to any source or destination for I/O that is treated as if it were a file (such as devices, files, directories, pipes, and mailslots). A typical user mode client calls the `CreateFile` or `CreateFile2` functions to get a handle to a virtual file. The function name is a little misleading—it's not just about files, it's anything that is known as a *symbolic link* within the object manager's directory called GLOBAL??. The suffix "File" in the `CreateFile*` functions really means a virtual file object (`FILE_OBJECT`) that is the entity created by the executive as a result of these functions. **Figure 6-2** shows a screenshot of the WinObj Sysinternals tool for the GLOBAL?? directory.

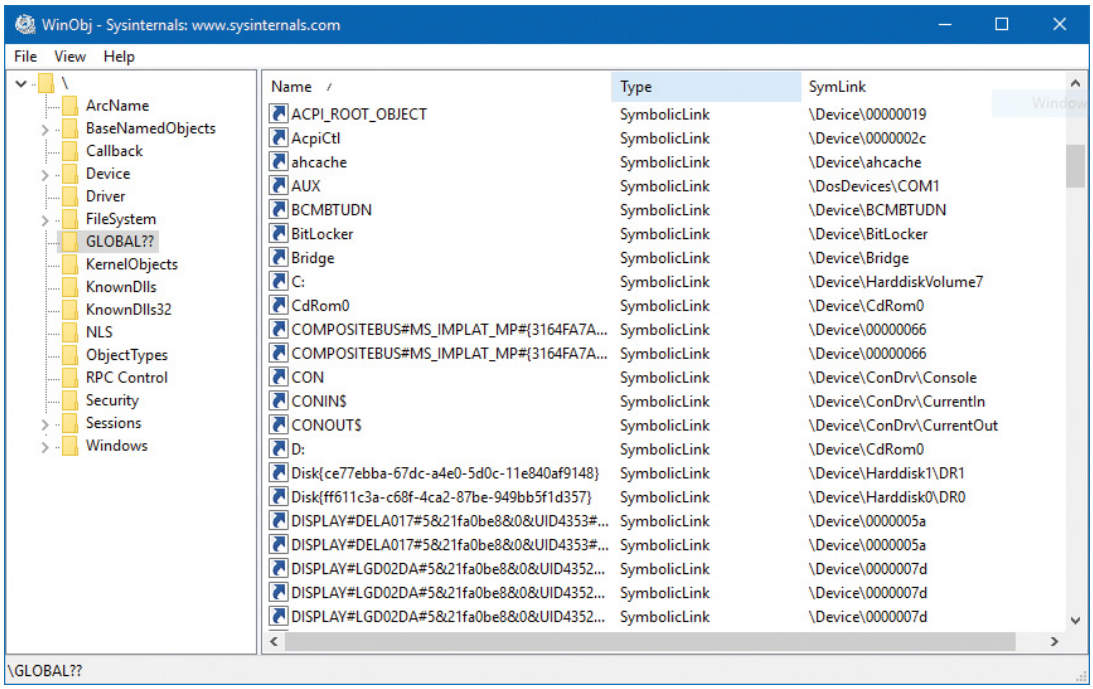


FIGURE 6-2 The object manager's GLOBAL?? directory.

As shown in **Figure 6-2**, a name such as C: is just a symbolic link to an internal name under the Device object manager directory (in this case, \Device\HarddiskVolume7). (See Chapter 8, "System mechanisms," in Part 2 for more on the object manager and the object manager namespace.) All the names in the GLOBAL?? directory are candidates for arguments to `CreateFile(2)`. Kernel mode clients such as device drivers can use the similar `ZwCreateFile` to obtain a handle to a virtual file.

NOTE

Higher-level abstractions such as the .NET Framework and the Windows Runtime have their own APIs for working with files and devices (for example, the `System.IO.File` class in .NET or the `Windows.Storage.StorageFile` class in WinRT), but these eventually call `CreateFile(2)` to get the actual handle they hide under the covers.

NOTE

The GLOBAL?? object manager directory is sometimes called DosDevices, which is an older name. `DosDevices` still works because it's defined as a symbolic link to *GLOBAL??* in the root of the object manager's namespace. In driver code, the `??` string is typically used to reference the GLOBAL?? directory.

The operating system abstracts all I/O requests as operations on a virtual file because the I/O manager has no knowledge of anything but files, therefore making it the responsibility of the driver to translate file-oriented comments (open, close, read, write) into device-specific commands. This abstraction thereby generalizes an application's interface to devices. User-mode applications call documented functions, which in turn call internal I/O system functions to read from a file, write to a file, and perform other operations. The I/O manager dynamically directs these virtual file requests to the appropriate device driver. **Figure 6-3** illustrates the basic structure of a typical I/O read request flow. (Other types of I/O requests, such as write, are similar; they just use different APIs.)

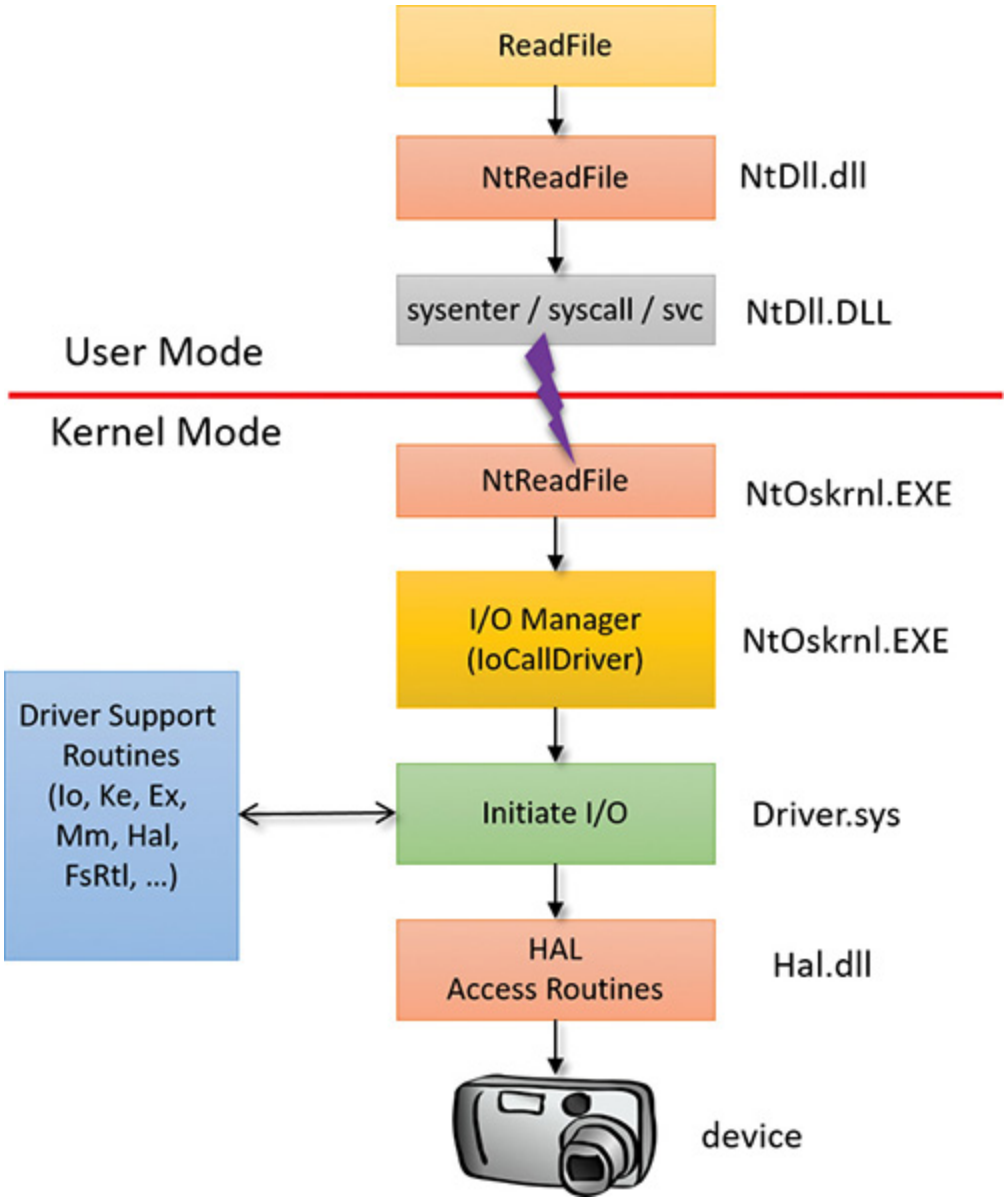


FIGURE 6-3 The flow of a typical I/O request.

The following sections look at these components more closely, covering the various types of device drivers, how they are structured, how they load and initialize, and how they process I/O requests. Then we'll cover the operation and roles of the PnP manager and the power manager.

Interrupt Request Levels and Deferred Procedure Calls

Before we proceed, we must introduce two very important concepts of the Windows kernel that play an important role within the I/O system: Interrupt Request Levels (IRQL) and Deferred Procedure Calls (DPC). A thorough discussion of these concepts is reserved for Chapter 8 in Part 2, but we'll provide enough information in this section to enable you to understand the mechanics of I/O processing that follow.

Interrupt Request Levels

The IRQL has two somewhat distinct meanings, but they converge in certain situations:

- **An IRQL is a priority assigned to an interrupt source from a hardware device** This number is set by the HAL (in conjunction with the interrupt controller to which devices that require interrupt servicing are connected).

- **Each CPU has its own IRQL value** It should be considered a register of the CPU (even though current CPUs do not implement it as such).

The fundamental rule of IRQLs is that lower IRQL code cannot interfere with higher IRQL code and vice versa—code with a higher IRQL can preempt code running at a lower IRQL. You'll see examples of how this works in practice in a moment. A list of IRQLs for the Windows-supported architectures is shown in **Figure 6-4**. Note that IRQLs are not the same as thread priorities. In fact, thread priorities have meaning only when the IRQL is less than 2.

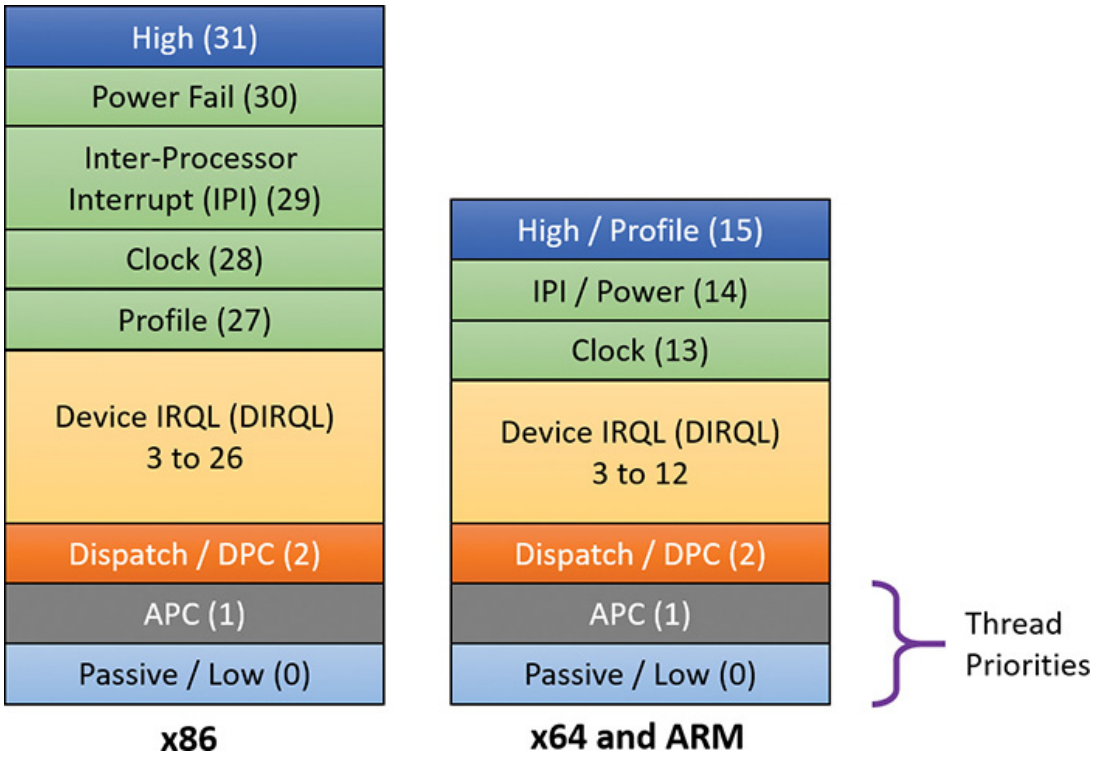


FIGURE 6-4 IRQLs.

NOTE

IRQL is not the same as IRQ (interrupt request). IRQs are hardware lines connecting devices to an interrupt controller. See Chapter 8 in Part 2 for more on interrupts, IRQs, and IRQLs.

Normally, the IRQL of a processor is 0. This means "nothing special" is happening in that regard, and that the kernel's scheduler that schedules threads based on priorities and so on works as described in **Chapter 4**, "**Threads**." In user mode, the IRQL can only be 0. There is no way to raise IRQL from user mode. (That's why user-mode documentation never mentions the IRQL concept at all; there would be no point.)

Kernel-mode code can raise and lower the current CPU IRQL with the `KeRaiseIrql` and `KeLowerIrql` functions. However, most of the time-specific functions are called with the IRQL raised to some expected level, as you'll see shortly when we discuss a typical I/O processing by a driver.

The most important IRQLs for this I/O-related discussions are the following:

■ **Passive(0)** This is defined by the `PASSIVE_LEVEL` macro in the WDK header wdm.h. It is the normal IRQL where the kernel scheduler is working normally, as described at length in **Chapter 4**.

■ **Dispatch/DPC (2) (`DISPATCH_LEVEL`)** This is the IRQL the kernel's scheduler works at. This means if a thread raises the current IRQL to 2 (or higher), the thread has essentially an infinite quantum and will not be preempted by another thread. Effectively, the scheduler cannot wake up on the current CPU until the IRQL drops below 2. This implies a few things:

• With the IRQL at level 2 or above, any waiting on kernel dispatcher objects (such as mutexes, semaphores, and events) would crash the sys-

tem. This is because waiting implies that the thread might enter a wait state and another should be scheduled on the same CPU. However, because the scheduler is not around at this level, this cannot happen; instead, the system will bug-check (the only exception is if the wait timeout is zero, meaning no waiting is requested, just getting back the signaled state of the object).

• No page faults can be handled. This is because a page fault would require a context switch to one of the modified page writers. However, context switches are not allowed, so the system would crash. This means code running at IRQL 2 or above can access only non-paged memory—typically memory allocated from non-paged pool, which by definition is always resident in physical memory.

■ **Device IRQL (3–26 on x86; 3–12 on x64 and ARM) (`DIRQL`)** These are the levels assigned to hardware interrupts. When an interrupt arrives, the kernel's trap dispatcher calls the appropriate interrupt service routine (ISR) and raises its IRQL to that of the associated interrupt. Because this value is always higher than `DISPATCH_LEVEL (2)`, all rules associated with IRQL 2 apply for `DIRQL` as well.

Running at a particular IRQL masks interrupts with that and lower IRQLs. For example, an ISR running with IRQL of 8 would not let any code interfere (on that CPU) with IRQL of 7 or lower. Specifically, no user mode code is able to run because it always runs at IRQL 0. This implies that running in high IRQL is not desirable in the general case; there are a few specific scenarios (which we'll look at in this chapter) where this makes sense and is in fact required for normal system operation.

Deferred Procedure Calls

A Deferred Procedure Call (DPC) is an object that encapsulates calling a function at IRQL `DPC_LEVEL (2)`. DPCs exist primarily for post-interrupt processing because running at DIRQL masks (and thus delays) other interrupts waiting to be serviced. A typical ISR would do the minimum work possible, mostly reading the state of the device and telling it to stop its interrupt signal and then deferring further processing to a lower IRQL (2) by requesting a DPC. The term *Deferred* means the DPC will not execute immediately—it can't because the current IRQL is higher than 2. However, when the ISR returns, if there are no pending interrupts waiting to be serviced, the CPU IRQL will drop to 2 and it will execute the DPCs that have accumulated (maybe just one). **Figure 6-5** shows a simplified example of the sequence of events that may occur when interrupts from hardware devices (which are asynchronous in nature, meaning they can arrive at any time) occur while code executes normally at IRQL 0 on some CPU.

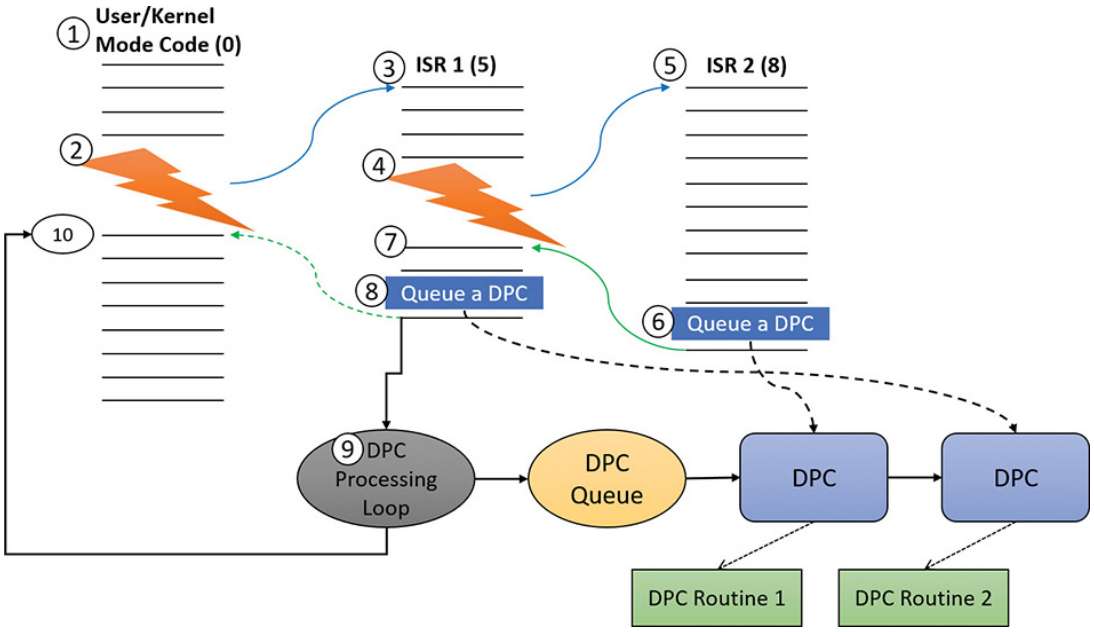


FIGURE 6-5 Example of interrupt and DPC processing.

Here is a rundown of the sequence of events shown in **Figure 6-5**:

1. Some user-mode or kernel-mode code is executing while the CPU is at IRQL 0, which is the case most of the time.

2. A hardware interrupt arrives with an IRQL of 5 (remember that Device IRQLs have a minimum value of 3). Because 5 is greater than zero (the current IRQL), the CPU state is saved, the IRQL is raised to 5, and the ISR associated with that interrupt is called. Note that there is no context switch; it's the same thread that now happens to execute the ISR code. (If the thread was in user mode, it switches to kernel mode whenever an interrupt arrives.)

3. ISR 1 starts executing while the CPU IRQL is 5. At this point, any interrupt with IRQL 5 or lower cannot interrupt.

4. Suppose another interrupt arrives with an IRQL of 8. Assume the system decides that the same CPU should handle it. Because 8 is greater than 5, the code is interrupted again, the CPU state is saved, the IRQL is raised to 8, and the CPU jumps to ISR 2. Note again that it's the same thread. No context switch can happen because the thread scheduler cannot wake up if the IRQL is 2 or higher.

5. ISR 2 is executing. Before it's done, ISR 2 would like to do some more processing at a lower IRQL so that interrupts with IRQLs less than 8 could be services as well.

6. As its final act, ISR 2 inserts a DPC initialized properly to point to a driver routine to do any post processing after the interrupt is dismissed by calling the `KeInsertQueueDpc` function. (We'll discuss what this post-processing typically includes in the next section.) Then the ISR returns, restoring the CPU state saved before entering ISR 2.

7. At this point, the IRQL drops to its previous level (5) and the CPU continues execution of ISR 1 that was interrupted before.

8. Just before ISR 1 finishes, it queues a DPC of its own to do its required post-processing. These DPCs are collected in a DPC queue that has not been examined yet. Then ISR 1 returns, restoring the CPU state saved before ISR 1 started execution.

9. At this point, the IRQL would want to drop to the old value of zero before all the interrupt handling began. However, the kernel notices that there are DPCs pending and so drops the IRQL to level 2 (`DPC_LEVEL`) and enters a DPC processing loop that iterates over the accumulated DPCs and calls each DPC routine in sequence. When the DPC queue is empty, DPC processing ends.

10. Finally, the IRQL can drop back to zero, restore the state of the CPU again, and resume execution of the original user or kernel code that got interrupted in the first place. Again, notice that all the processing described was done by the same thread (whichever one that may be). This fact implies that ISRs and DPC routines should not rely on any particular thread (and hence part of a particular process) to execute their code. It could be any thread, the significance of which will be discussed in the next section.

The preceding description is a bit simplified. It doesn't mention DPC importance, other CPUs that may handle DPCs for quicker DPC processing, and more. These details are not important for the discussion in this chapter. However, they are described fully in Chapter 8 in Part 2.

Device drivers

To integrate with the I/O manager and other I/O system components, a device driver must conform to implementation guidelines specific to the type of device it manages and the role it plays in managing the device. This section discusses the types of device drivers Windows supports as well as the internal structure of a device driver.

NOTE

Most kernel-mode device drivers are written in C. Starting with the Windows Driver Kit 8.0, drivers can also be safely written in C++ due to specific support for kernel-mode C++ in the new compilers. Use of assembly language is highly discouraged because of the complexity it introduces and its effect of making a driver difficult to port between the hardware architectures supported by Windows (x86, x64, and ARM).

Types of device drivers

Windows supports a wide range of device-driver types and programming environments. Even within a particular type of device driver, programming environments can differ depending on the specific type of device for which a driver is intended.

The broadest classification of a driver is whether it is a user-mode or kernel-mode driver. Windows supports a couple of types of user-mode drivers:

- **Windows subsystem printer drivers** These translate device-independent graphics requests to printer-specific commands. These commands are then typically forwarded to a kernel-mode port driver such as the universal serial bus (USB) printer port driver (Usbprint.sys).

- **User-Mode Driver Framework (UMDF) drivers** These are hardware device drivers that run in user mode. They communicate to the kernel-mode UMDF support library through advanced local procedure calls (ALPC). See the "**User-Mode Driver Framework**" section later in this chapter for more information.

In this chapter, the focus is on kernel-mode device drivers. There are many types of kernel-mode drivers, which can be divided into the following basic categories:

■ **File-system drivers** These accept I/O requests to files and satisfy the requests by issuing their own more explicit requests to mass storage or network device drivers.

■ **Plug and Play drivers** These work with hardware and integrate with the Windows power manager and PnP manager. They include drivers for mass storage devices, video adapters, input devices, and network adapters.

■ **Non–Plug and Play drivers** These include kernel extensions, which are drivers or modules that extend the functionality of the system. They do not typically integrate with the PnP manager or power manager because they usually do not manage an actual piece of hardware. Examples include network API and protocol drivers. The Sysinternals tool Process Monitor has a driver, and is an example of a non-PnP driver.

Within the category of kernel-mode drivers are further classifications based on the driver model to which the driver adheres and its role in servicing device requests.

WDM drivers

WDM drivers are device drivers that adhere to the Windows Driver Model (WDM). WDM includes support for Windows power management, Plug and Play, and WMI, and most Plug and Play drivers adhere to WDM. There are three types of WDM drivers:

■ **Bus drivers** These manage a logical or physical bus. Examples of buses include PCMCIA, PCI, USB, and IEEE 1394. A bus driver is responsible for detecting and informing the PnP manager of devices attached to the bus it controls and for managing the power setting of the bus. These are typically provided by Microsoft out of the box.

■ **Function drivers** These manage a particular type of device. Bus drivers present devices to function drivers via the PnP manager. The function driver is the driver that exports the operational interface of the de-

vice to the operating system. In general, it's the driver with the most knowledge about the operation of the device.

■ **Filter drivers** These logically layer either above function drivers (these are called *upper filters* or *function filters*) or above the bus driver (these are called *lower filters* or *bus filters*), augmenting or changing the behavior of a device or another driver. For example, a keyboard-capture utility could be implemented with a keyboard filter driver that layers above the keyboard function driver.

Figure 6-6 shows a device node (also called a *devnode*) with a bus driver that creates a physical device object (PDO), lower filters, a function driver that creates a functional device object (FDO), and upper filters. The only required layers are the PDO and FDO. The various filters may or may not exist.

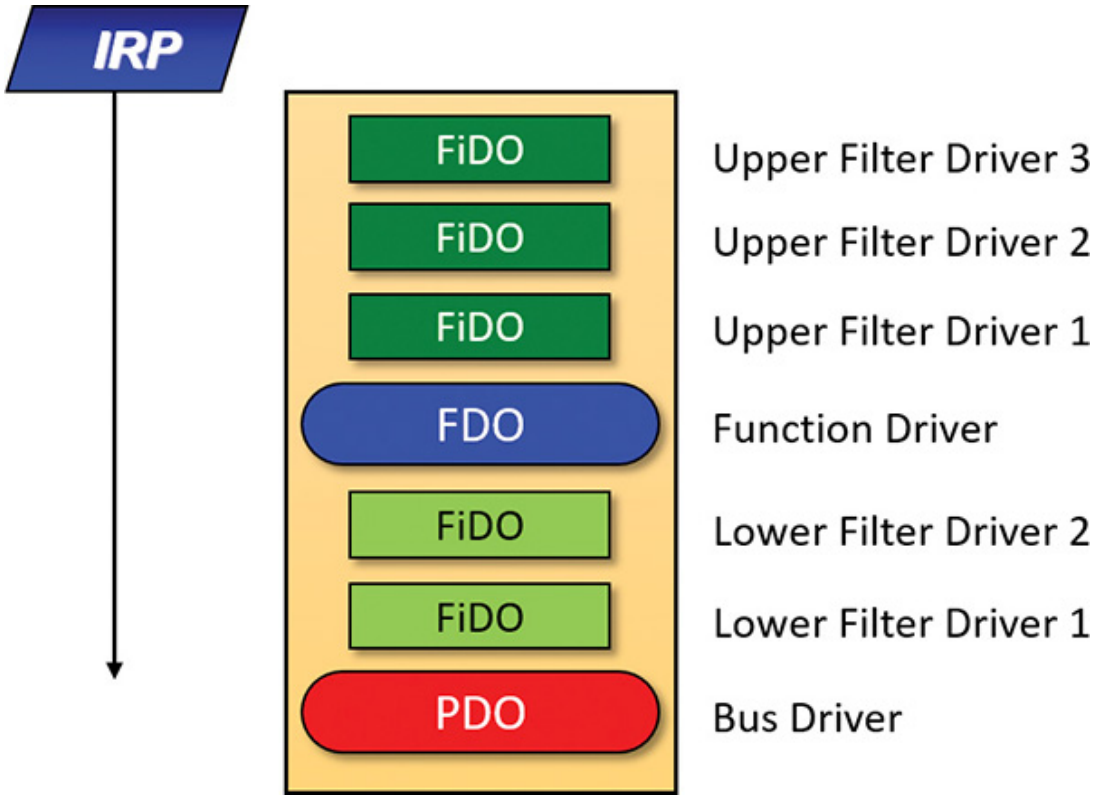


FIGURE 6-6 WDM device node (devnode).

In WDM, no one driver is responsible for controlling all aspects of a particular device. The bus driver is responsible for detecting bus membership changes (device addition or removal), assisting the PnP manager in enumerating the devices on the bus, accessing bus-specific con-

figuration registers, and, in some cases, controlling power to devices on the bus. The function driver is generally the only driver that accesses the device's hardware. The exact manner in which these devices came to be is described in "**The Plug and Play manager**" section later in this chapter.

Layered drivers

Support for an individual piece of hardware is often divided among several drivers, each providing a part of the functionality required to make the device work properly. In addition to WDM bus drivers, function drivers, and filter drivers, hardware support might be split between the following components:

- **Class drivers** These implement the I/O processing for a particular class of devices, such as disk, keyboard, or CD-ROM, where the hardware interfaces have been standardized so one driver can serve devices from a wide variety of manufacturers.

- **Miniclass drivers** These implement I/O processing that is vendor-defined for a particular class of devices. For example, although Microsoft has written a standardized battery class driver, both uninterruptible power supplies (UPS) and laptop batteries have highly specific interfaces that differ wildly between manufacturers, such that a miniclass is required from the vendor. *Miniclass drivers* are essentially kernel-mode DLLs and do not perform IRP processing directly. Instead, the class driver calls into them and they import functions from the class driver.

- **Port drivers** These implement the processing of an I/O request specific to a type of I/O port, such as SATA, and are implemented as kernel-mode libraries of functions rather than actual device drivers. Port drivers are almost always written by Microsoft because the interfaces are typically standardized in such a way that different vendors can still share the same port driver. However, in certain cases, third parties may need to write their own for specialized hardware. In some cases, the concept of I/O port extends to cover logical ports as well. For example,

Network Driver Interface Specification (NDIS) is the network "port" driver.

- **Miniport drivers** These map a generic I/O request to a type of port into an adapter type, such as a specific network adapter. Miniport drivers are actual device drivers that import the functions supplied by a port driver. Miniport drivers are written by third parties, and they provide the interface for the port driver. Like miniclass drivers, they are kernel-mode DLLs and do not perform IRP processing directly.

Figure 6-7 shows a simplified example for illustrative purposes that will help demonstrate how device drivers and layering work at a high level. As you can see, a file-system driver accepts a request to write data to a certain location within a particular file. It translates the request into a request to write a certain number of bytes to the disk at a particular (that is, the logical) location. It then passes this request (via the I/O manager) to a simple disk driver. The disk driver, in turn, translates the request into a physical location on the disk and communicates with the disk to write the data.

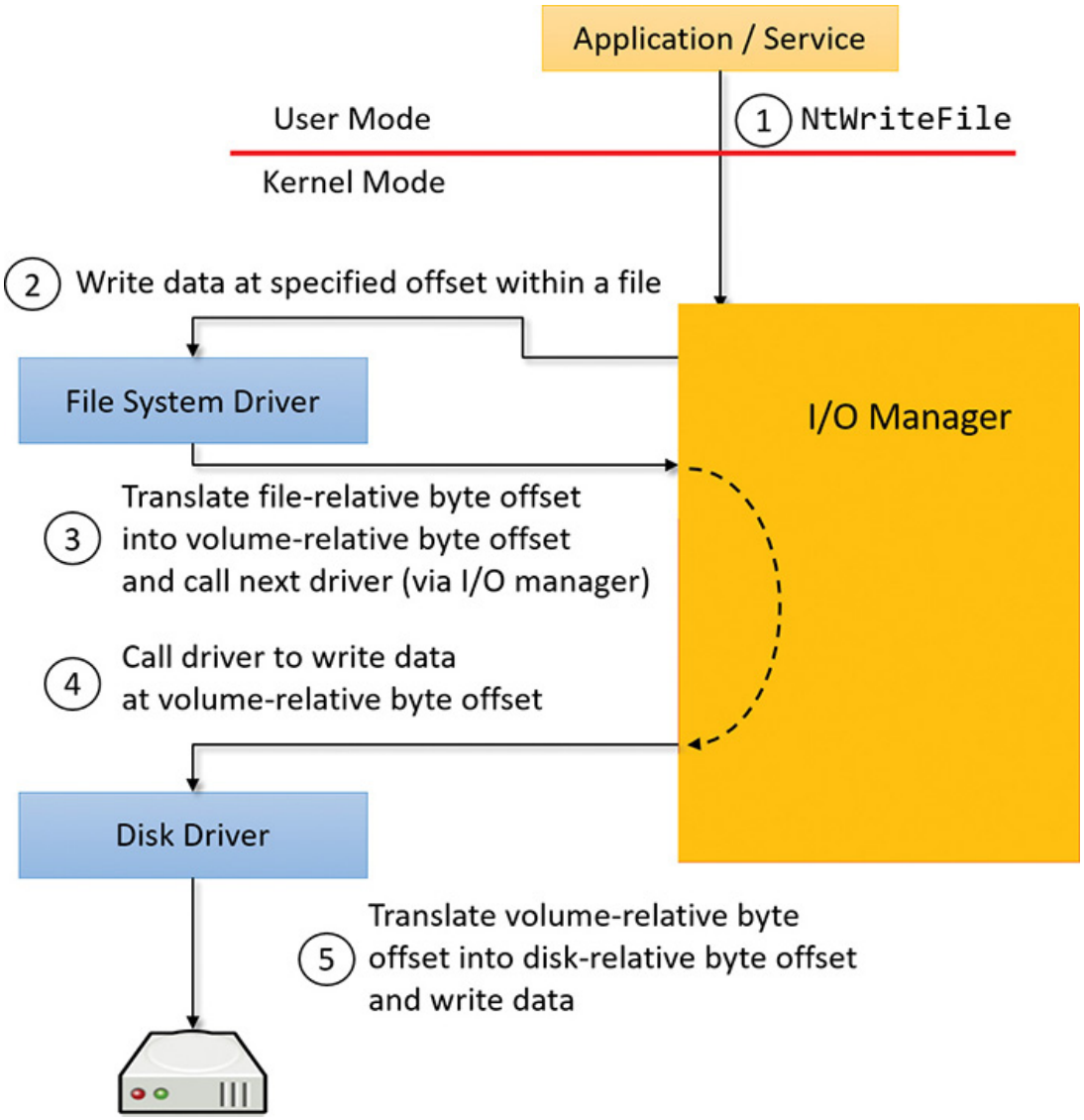


FIGURE 6-7 Layering of a file-system driver and a disk driver.

This figure illustrates the division of labor between two layered drivers. The I/O manager receives a write request that is relative to the beginning of a particular file. The I/O manager passes the request to the file-system driver, which translates the write operation from a file-relative operation to a starting location (a sector boundary on the disk) and a number of bytes to write. The file-system driver calls the I/O manager to pass the request to the disk driver, which translates the request to a physical disk location and transfers the data.

Because all drivers—both device drivers and file-system drivers—present the same framework to the operating system, another driver can easily be inserted into the hierarchy without altering the existing drivers or the I/O system. For example, several disks can be made to seem like a very large single disk by adding a driver. This logical vol-

ume manager driver is located between the file system and the disk drivers, as shown in the conceptual simplified architectural diagram presented in **Figure 6-8**. (For the actual storage driver stack diagram as well as volume manager drivers, see Chapter 12, "Storage management" in Part 2.)

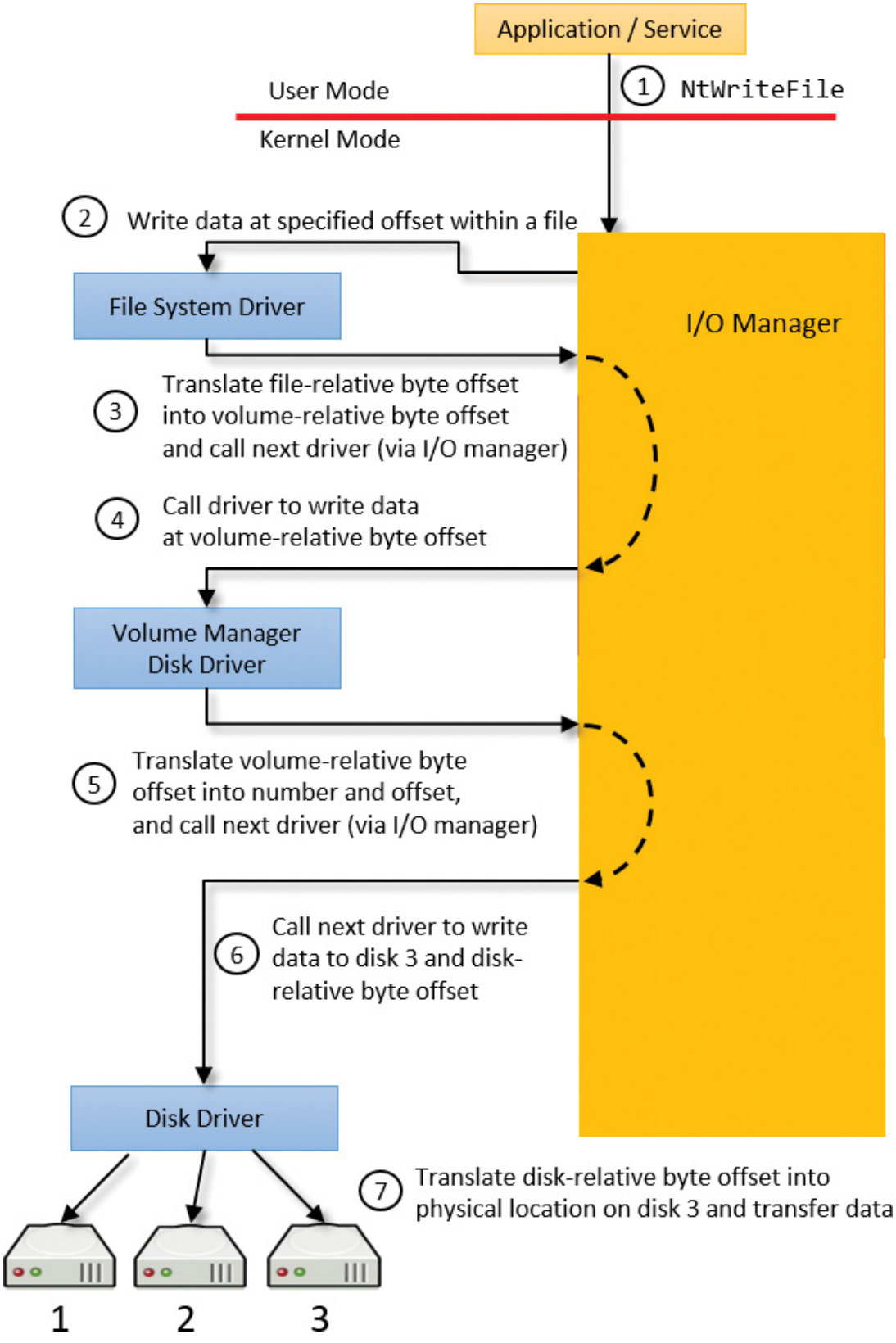
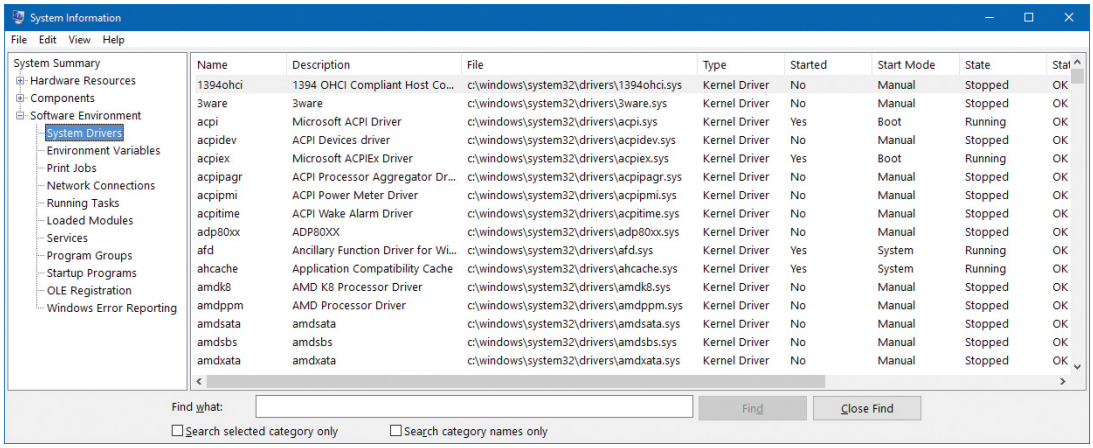


FIGURE 6-8 Adding a layered driver.

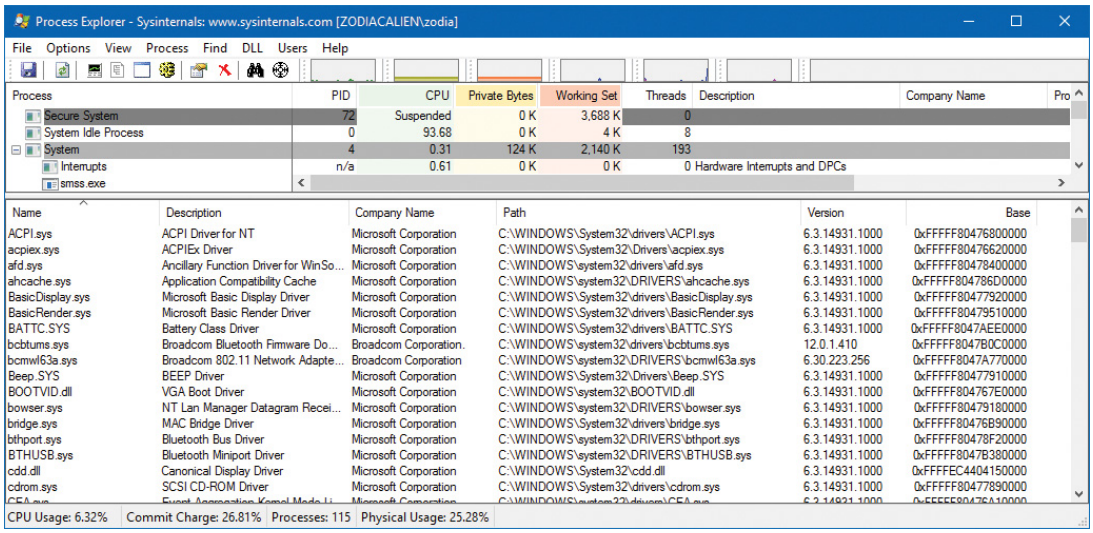
EXPERIMENT: VIEWING THE LOADED DRIVER LIST

You can see a list of registered drivers by executing the Msinfo32.exe utility from the Run dialog box, accessible from the Start menu. Select the **System Drivers** entry under **Software Environment** to see the list of drivers configured on the system. Those that are loaded contain the text *Yes* in the Started column, as shown here:



The list of drivers comes from the registry subkeys under HKLM\System\CurrentControlSet\Services. This key is shared between drivers and services. Both can be started by the Service Control Manager (SCM). The way to distinguish between a driver and a service for each subkey is by looking at the `Type` value. A small value (`1`, `2`, `4`, `8`) indicates a driver, while `16 (0x10)` and `32 (0x20)` indicate a Windows service. For more information on the Services subkey, consult Chapter 9 in Part 2.

You can also view the list of loaded kernel-mode drivers with Process Explorer. Run Process Explorer, select the **System** process, and select **DLLs** from the **Lower Pane View** menu entry in the **View** menu:



Process Explorer lists the loaded drivers, their names, version information (including company and description), and load address (assuming you have configured Process Explorer to display the corresponding columns).

Finally, if you're looking at a crash dump (or live system) with the kernel debugger, you can get a similar display with the kernel debugger `lm kv` command:

Click here to view code image

```
kd> lm kv

start    end        module name

80626000 80631000   kdcom      (deferred)

    Image path: kdcom.dll

    Image name: kdcom.dll

    Browse all global symbols  functions  data

    Timestamp:        Sat Jul 16 04:27:27 2016 (57898D7F)

    CheckSum:         0000821A

    ImageSize:        0000B000

    Translations:     0000.04b0 0000.04e4 0409.04b0 0409.04e4
```

```
81009000 81632000   nt         (pdb symbols)          e:\symbols\ntkrpamp.
pdb\A54DF85668E54895982F873F58C984591\ntkrpamp.pdb
    Loaded symbol image file: ntkrpamp.exe
    Image path: ntkrpamp.exe
    Image name: ntkrpamp.exe
    Browse all global symbols  functions  data
    Timestamp:        Wed Sep 07 07:35:39 2016 (57CF991B)
    CheckSum:         005C6B08
    ImageSize:        00629000
    Translations:     0000.04b0 0000.04e4 0409.04b0 0409.04e4
81632000 81693000   hal        (deferred)
    Image path: halmacpi.dll
    Image name: halmacpi.dll
    Browse all global symbols  functions  data
    Timestamp:        Sat Jul 16 04:27:33 2016 (57898D85)
    CheckSum:         00061469
    ImageSize:        00061000
    Translations:     0000.04b0 0000.04e4 0409.04b0 0409.04e4
8a800000 8a84b000   FLTMGR     (deferred)
    Image path: \SystemRoot\System32\drivers\FLTMGR.SYS
    Image name: FLTMGR.SYS
```

```
    Browse all global symbols  functions  data

    Timestamp:        Sat Jul 16 04:27:37 2016 (57898D89)

    CheckSum:         00053B90

    ImageSize:        0004B000

    Translations:     0000.04b0 0000.04e4 0409.04b0 0409.04e4

...
```

Structure of a driver

The I/O system drives the execution of device drivers. Device drivers consist of a set of routines that are called to process the various stages of an I/O request. **Figure 6-9** illustrates the key driver-function routines, which are described next.

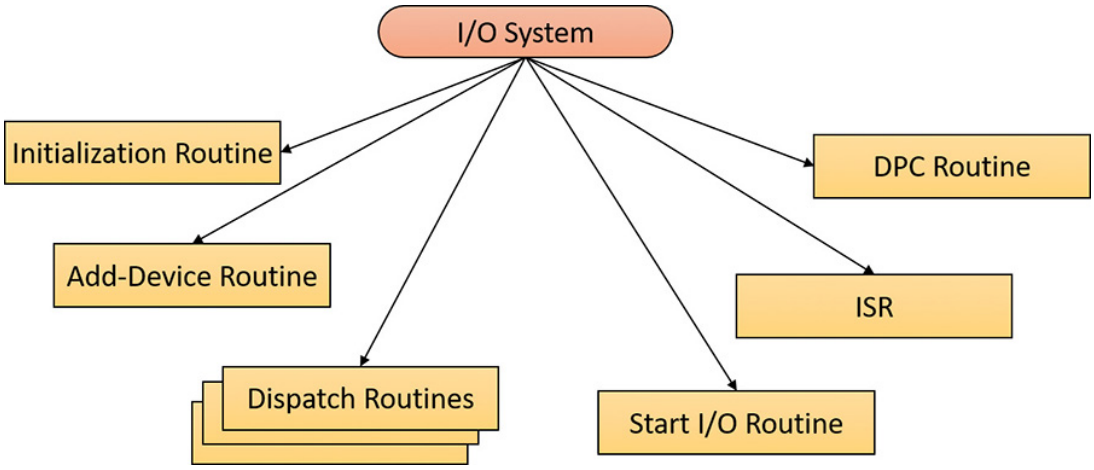


FIGURE 6-9 Primary device driver routines.

■ **An initialization routine** The I/O manager executes a driver's initialization routine, which is set by the WDK to `GSDriverEntry` when it loads the driver into the operating system. `GSDriverEntry` initializes the compiler's protection against stack-overflow errors (called a *cookie*) and then calls `DriverEntry`, which is what the driver writer must implement. The routine fills in system data structures to register the rest

of the driver's routines with the I/O manager and performs any necessary global driver initialization.

■ **An add-device routine** A driver that supports Plug and Play implements an add-device routine. The PnP manager sends a notification to the driver via this routine whenever a device for which the driver is responsible is detected. In this routine, a driver typically creates a device object (described later in this chapter) to represent the device.

■ **A set of dispatch routines** Dispatch routines are the main entry points that a device driver provides. Some examples are open, close, read, write, and Plug and Play. When called on to perform an I/O operation, the I/O manager generates an IRP and calls a driver through one of the driver's dispatch routines.

■ **A start I/O routine** A driver can use a start I/O routine to initiate a data transfer to or from a device. This routine is defined only in drivers that rely on the I/O manager to queue their incoming I/O requests. The I/O manager serializes IRPs for a driver by ensuring that the driver processes only one IRP at a time. Drivers can process multiple IRPs concurrently, but serialization is usually required for most devices because they cannot concurrently handle multiple I/O requests.

■ **An interrupt service routine (ISR)** When a device interrupts, the kernel's interrupt dispatcher transfers control to this routine. In the Windows I/O model, ISRs run at device interrupt request level (DIRQL), so they perform as little work as possible to avoid blocking lower IRQL interrupts (as discussed in the previous section). An ISR usually queues a DPC, which runs at a lower IRQL (DPC/dispatch level) to execute the remainder of interrupt processing. Only drivers for interrupt-driven devices have ISRs; a file-system driver, for example, doesn't have one.

■ **An interrupt-servicing DPC routine** A DPC routine performs most of the work involved in handling a device interrupt after the ISR executes. The DPC routine executes at IRQL 2, which is a "compromise" between the high DIRQL and the low passive level (0). A typical DPC routine initi-

ates I/O completion and starts the next queued I/O operation on a device.

Although the following routines aren't shown in **Figure 6-9**, they're found in many types of device drivers:

- **One or more I/O completion routines** A layered driver might have I/O completion routines that notify it when a lower-level driver finishes processing an IRP. For example, the I/O manager calls a file-system driver's I/O completion routine after a device driver finishes transferring data to or from a file. The completion routine notifies the file-system driver about the operation's success, failure, or cancellation, and allows the file-system driver to perform cleanup operations.

- **A cancel I/O routine** If an I/O operation can be canceled, a driver can define one or more cancel I/O routines. When the driver receives an IRP for an I/O request that can be canceled, it assigns a cancel routine to the IRP. As the IRP goes through various stages of processing, this routine can change or outright disappear if the current operation is not cancellable. If a thread that issues an I/O request exits before the request is completed or the operation is cancelled (for example, with the `CancelIo` or `CancelIoEx` Windows functions), the I/O manager executes the IRP's cancel routine if one is assigned to it. A cancel routine is responsible for performing whatever steps are necessary to release any resources acquired during the processing that has already taken place for the IRP as well as for completing the IRP with a canceled status.

- **Fast-dispatch routines** Drivers that make use of the cache manager, such as file-system drivers, typically provide these routines to allow the kernel to bypass typical I/O processing when accessing the driver. (See Chapter 14, "Cache manager," in Part 2, for more information on the cache manager.) For example, operations such as reading or writing can be quickly performed by accessing the cached data directly instead of taking the I/O manager's usual path that generates discrete I/O operations. Fast dispatch routines are also used as a mechanism for callbacks from the memory manager and cache manager to file-system drivers.

For instance, when creating a section, the memory manager calls back into the file-system driver to acquire the file exclusively.

■ **An unload routine** An unload routine releases any system resources a driver is using so that the I/O manager can remove the driver from memory. Any resources acquired in the initialization routine (`DriverEntry`) are usually released in the unload routine. A driver can be loaded and unloaded while the system is running if the driver supports it, but the unload routine will be called only after all file handles to the device are closed.

■ **A system shutdown notification routine** This routine allows driver cleanup on system shutdown.

■ **Error-logging routines** When unexpected errors occur (for example, when a disk block goes bad), a driver's error-logging routines note the occurrence and notify the I/O manager. The I/O manager then writes this information to an error log file.

Driver objects and device objects

When a thread opens a handle to a file object (described in the "**I/O processing**" section later in this chapter), the I/O manager must determine from the file object's name which driver it should call to process the request. Furthermore, the I/O manager must be able to locate this information the next time a thread uses the same file handle. The following system objects fill this need:

■ **A driver object** This represents an individual driver in the system (`DRIVER_OBJECT` structure). The I/O manager obtains the address of each of the driver's dispatch routines (entry points) from the driver object.

■ **A device object** This represents a physical or logical device on the system and describes its characteristics (`DEVICE_OBJECT` structure), such as the alignment it requires for buffers and the location of its de-

vice queue to hold incoming IRPs. It is the target for all I/O operations because this object is what the handle communicates with.

The I/O manager creates a driver object when a driver is loaded into the system. It then calls the driver's initialization routine (`DriverEntry`), which fills in the object attributes with the driver's entry points.

At any time after loading, a driver creates device objects to represent logical or physical devices—or even a logical interface or endpoint to the driver—by calling `IoCreateDevice` or `IoCreateDevice-Secure`. However, most Plug and Play drivers create devices in their add-device routine when the PnP manager informs them of the presence of a device for them to manage. Non–Plug and Play drivers, on the other hand, usually create device objects when the I/O manager invokes their initialization routine. The I/O manager unloads a driver when the driver's last device object has been deleted and no references to the driver remain.

The relationship between a driver object and its device objects is shown in **Figure 6-10**.

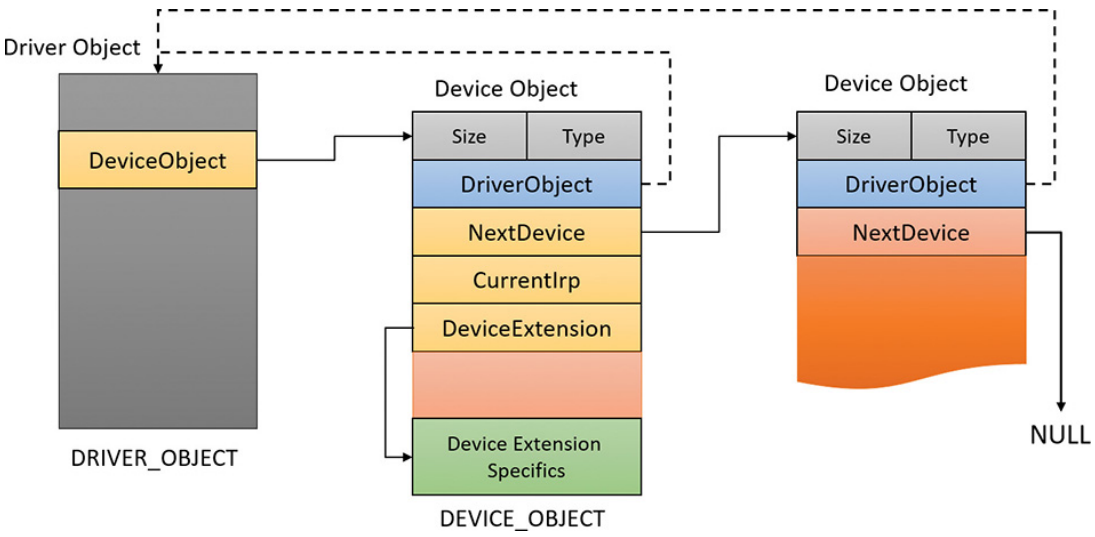


FIGURE 6-10 A driver object and its device objects.

A driver object holds a pointer to its first device object in the `DeviceObject` member. The second device object is pointed to by the `NextDevice` member of `DEVICE_OBJECT` until the last one points to `NULL`. Each device object points back to its driver object with the

`DriverObject` member. All the arrows shown in **Figure 6-10** are built by the device-creation functions (`IoCreateDevice` or `IoCreateDeviceSecure`). The `DeviceExtension` pointer shown is a way a driver can allocate an extra piece of memory that is attached to each device object it manages.

NOTE

It's important to distinguish driver objects from device objects. A driver object represents the behavior of a driver, while individual device objects represent communication endpoints. For example, on a system with four serial ports, there would be one driver object (and one driver binary) but four instances of device objects, each representing a single serial port, that can be opened individually with no effect on the other serial ports. For hardware devices, each device also represents a distinct set of hardware resources, such as I/O ports, memory-mapped I/O, and interrupt line. Windows is device-centric, rather than driver-centric.

When a driver creates a device object, the driver can optionally assign the device a name. A name places the device object in the object manager namespace. A driver can either explicitly define a name or let the I/O manager auto-generate one. By convention, device objects are placed in the \Device directory in the namespace, which is inaccessible by applications using the Windows API.

NOTE

Some drivers place device objects in directories other than \Device. For example, the IDE driver creates the device objects that represent IDE ports and channels in the \Device\Ide directory. See Chapter 12 in Part 2 for a description of storage architecture, including the way storage drivers use device objects.

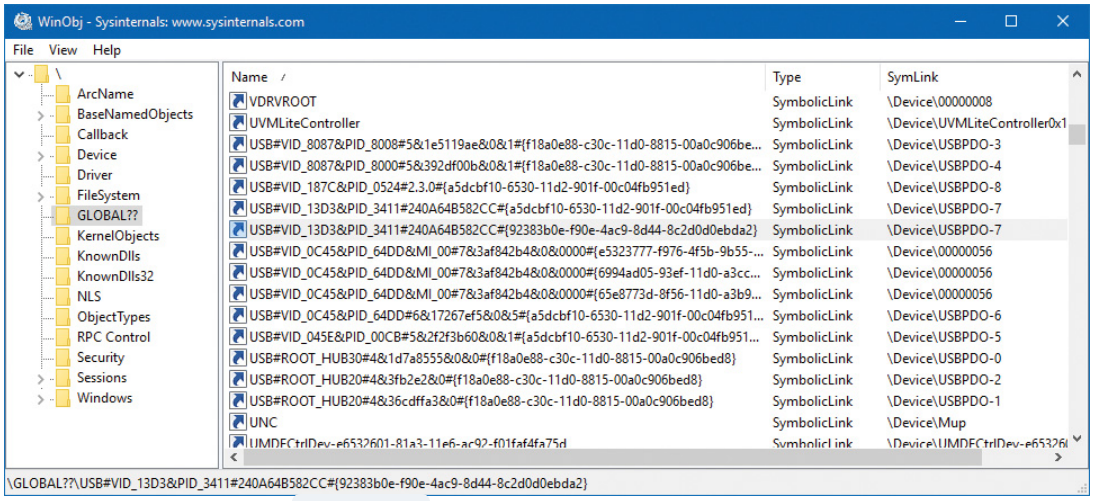
If a driver needs to make it possible for applications to open the device object, it must create a symbolic link in the \GLOBAL?? directory to the device object's name in the \Device directory. (The `IoCreateSymbolicLink` function accomplishes this.) Non–Plug and Play and file-system drivers typically create a symbolic link with a well-known name (for example, \Device\HarddiskVolume2). Because well-known names don't work well in an environment in which hardware appears and disappears dynamically, PnP drivers expose one or more interfaces by calling the `IoRegisterDeviceInterface` function, specifying a globally unique identifier (GUID) that represents the type of functionality exposed. GUIDs are 128-bit values that can be generated by using tools such as `uuidgen` and `guidgen`, which are included with the WDK and the Windows SDK. Given the range of values that 128 bits represents (and the formula used to generate them), it's statistically almost certain that each GUID generated will be forever and globally unique.

`IoRegisterDeviceInterface` generates the symbolic link associated with a device instance. However, a driver must call `IoSetDeviceInterfaceState` to enable the interface to the device before the I/O manager actually creates the link. Drivers usually do this when the PnP manager starts the device by sending the driver a start-device IRP—in this case, `IRP_MJ_PNP` (major function code) with `IRP_MN_START_DEVICE` (minor function code). IRPs are discussed in the "**I/O request packets**" section later in this chapter.

An application that wants to open a device object whose interfaces are represented with a GUID can call Plug and Play setup functions in user space, such as `SetupDiEnumDeviceInterfaces`, to enumerate the interfaces present for a particular GUID and to obtain the names of the symbolic links it can use to open the device objects. For each device reported by `SetupDiEnumDeviceInterfaces`, the application executes `SetupDiGetDeviceInterfaceDetail` to obtain additional information about the device, such as its auto-generated name. After obtaining a device's name from `SetupDiGetDeviceInterface- Detail`, the application can execute the Windows function `CreateFile` or `CreateFile2` to open the device and obtain a handle.

EXPERIMENT: LOOKING AT DEVICE OBJECTS

You can use the WinObj tool from Sysinternals or the `!object` kernel debugger command to view the device names under \Device in the object manager namespace. The following screenshot shows an I/O manager–assigned symbolic link that points to a device object in \Device with an auto-generated name:



When you run the `!object` kernel debugger command and specify the \Device directory, you should see output similar to the following:

Click here to view code image

```
1: kd> !object \device
Object: 8200c530  Type: (8542b188) Directory
    ObjectHeader: 8200c518 (new version)
    HandleCount: 0  PointerCount: 231
    Directory Object: 82007d20  Name: Device

    Hash Address  Type                Name
    ---- -------  ----                ----
     00  d024a448 Device              NisDrv
         959afc08 Device              SrvNet
         958beef0 Device              WUDFLpcDevice
         854c69b8 Device              FakeVid1
         8befec98 Device              RdpBus
         88f7c338 Device              Beep
         89d64500 Device              Ndis
         8a24e250 SymbolicLink        ScsiPort2
```

```
    89d6c580 Device          KsecDD
    89c15810 Device          00000025
    89c17408 Device          00000019
01  854c6898 Device          FakeVid2
    88f98a70 Device          Netbios
    8a48c6a8 Device          NameResTrk
    89c2fe88 Device          00000026
02  854c6778 Device          FakeVid3
    8548fee0 Device          00000034
    8a214b78 SymbolicLink    Ip
    89c31038 Device          00000027
03  9c205c40 Device          00000041
    854c6658 Device          FakeVid4
    854dd9d8 Device          00000035
    8d143488 Device          Video0
    8a541030 Device          KeyboardClass0
    89c323c8 Device          00000028
    8554fb50 Device          KMDF0
04  958bb040 Device          ProcessManagement
    97ad9fe0 SymbolicLink    MailslotRedirector
    854f0090 Device          00000036
    854c6538 Device          FakeVid5
    8bf14e98 Device          Video1
    8bf2fe20 Device          KeyboardClass1
    89c332a0 Device          00000029
    89c05030 Device          VolMgrControl
    89c3a1a8 Device          VMBus
...
```

When you enter the `!object` command and specify an object manager directory object, the kernel debugger dumps the contents of the directory according to the way the object manager organizes it internally. For fast lookups, a directory stores objects in a hash table based on a hash of the object names, so the output shows the objects stored in each bucket of the directory's hash table.

As **Figure 6-10** illustrates, a device object points back to its driver object, which is how the I/O manager knows which driver routine to call when it receives an I/O request. It uses the device object to find the driver object representing the driver that services the device. It then indexes into the driver object by using the function code supplied in the original request. Each function code corresponds to a driver entry point (called a *dispatch routine*).

A driver object often has multiple device objects associated with it. When a driver is unloaded from the system, the I/O manager uses the queue of device objects to determine which devices will be affected by the removal of the driver.

EXPERIMENT: DISPLAYING DRIVER AND DEVICE OBJECTS

You can display driver and device objects with the `!drvobj` and `!devobj` kernel debugger commands, respectively. In the following example, the driver object for the keyboard class driver is examined, and one of its device objects viewed:

Click here to view code image

```
1: kd> !drvobj kbdclass
Driver object (8a557520) is for:
 \Driver\kbdclass
Driver Extension List: (id , addr)

Device Object list:
9f509648  8bf2fe20  8a541030
1: kd> !devobj 9f509648
Device object (9f509648) is for:
 KeyboardClass2 \Driver\kbdclass DriverObject 8a557520
Current Irp 00000000 RefCount 0 Type 0000000b Flags 00002044
Dacl 82090960 DevExt 9f509700 DevObjExt 9f5097f0
ExtensionFlags (0x00000c00)  DOE_SESSION_DEVICE,
DOE_DEFAULT_SD_PRESENT
Characteristics (0x00000100)  FILE_DEVICE_SECURE_OPEN
AttachedTo (Lower) 9f509848 \Driver\terminpt
Device queue is not busy.
```

Notice that the `!devobj` command also shows you the addresses and names of any device objects that the object you're viewing is layered over (the `AttachedTo` line). It can also show the device objects layered on top of the object specified (the `AttachedDevice` line), although not in this case.

The `!drvobj` command can accept an optional argument that indicates more information to show. Here is an example with the most information to show:

Click here to view code image

```
1: kd> !drvobj kbdclass 7
Driver object (8a557520) is for:
 \Driver\kbdclass
Driver Extension List: (id , addr)

Device Object list:
9f509648  8bf2fe20  8a541030

DriverEntry:   8c30a010     kbdclass!GsDriverEntry
DriverStartIo: 00000000
DriverUnload:  00000000
AddDevice:     8c307250     kbdclass!KeyboardAddDevice

Dispatch routines:
[00]
IRP_MJ_CREATE                   8c301d80       kbdclass!KeyboardClassCreate
[01]
IRP_MJ_CREATE_NAMED_PIPE        81142342       nt!IopInvalidDeviceRequest
[02]
IRP_MJ_CLOSE                    8c301c90       kbdclass!KeyboardClassClose
[03]
IRP_MJ_READ                     8c302150       kbdclass!KeyboardClassRead
[04]
IRP_MJ_WRITE                    81142342       nt!IopInvalidDeviceRequest
[05]
IRP_MJ_QUERY_INFORMATION        81142342       nt!IopInvalidDeviceRequest
[06]
IRP_MJ_SET_INFORMATION          81142342       nt!IopInvalidDeviceRequest
[07]
IRP_MJ_QUERY_EA                 81142342       nt!IopInvalidDeviceRequest
[08]
IRP_MJ_SET_EA                   81142342       nt!IopInvalidDeviceRequest
[09]
IRP_MJ_FLUSH_BUFFERS            8c303678       kbdclass!KeyboardClassFlush
```

[0a]
IRP_MJ_QUERY_VOLUME_INFORMATION 81142342 nt!IopInvalidDeviceRequest
[0b]
IRP_MJ_SET_VOLUME_INFORMATION 81142342 nt!IopInvalidDeviceRequest
[0c]
IRP_MJ_DIRECTORY_CONTROL 81142342 nt!IopInvalidDeviceRequest
[0d]
IRP_MJ_FILE_SYSTEM_CONTROL 81142342 nt!IopInvalidDeviceRequest
[0e]
IRP_MJ_DEVICE_CONTROL 8c3076d0 kbdclass!KeyboardClassDevice
Control
[0f]
IRP_MJ_INTERNAL_DEVICE_CONTROL 8c307ff0 kbdclass!KeyboardClassPass
Through
[10]
IRP_MJ_SHUTDOWN 81142342 nt!IopInvalidDeviceRequest
[11]
IRP_MJ_LOCK_CONTROL 81142342 nt!IopInvalidDeviceRequest
[12]
IRP_MJ_CLEANUP 8c302260 kbdclass!KeyboardClassCleanup
[13]
IRP_MJ_CREATE_MAILSLOT 81142342 nt!IopInvalidDeviceRequest
[14]
IRP_MJ_QUERY_SECURITY 81142342 nt!IopInvalidDeviceRequest
[15]
IRP_MJ_SET_SECURITY 81142342 nt!IopInvalidDeviceRequest
[16]
IRP_MJ_POWER 8c301440 kbdclass!KeyboardClassPower
[17]
IRP_MJ_SYSTEM_CONTROL 8c307f40 kbdclass!KeyboardClassSystem
Control
[18]
IRP_MJ_DEVICE_CHANGE 81142342 nt!IopInvalidDeviceRequest
[19]
IRP_MJ_QUERY_QUOTA 81142342 nt!IopInvalidDeviceRequest
[1a]

IRP_MJ_SET_QUOTA 81142342 nt!IopInvalidDeviceRequest
[1b] IRP_MJ_PNP 8c301870 kbdclass!KeyboardPnP

The dispatch routines array is clearly shown, and will be discussed in the next section. Note that operations that are not supported by the driver point to an I/O manager's routine `IopInvalidDeviceRequest`.

The address to the `!drvobj` command is for a `DRIVER_OBJECT` structure, and the address for the `!devobj` command is for a `DEVICE_OBJECT`. You can view these structures directly using the debugger:

Click here to view code image

```
1: kd> dt nt!_driver_object 8a557520
   +0x000 Type           : 0n4
   +0x002 Size           : 0n168
   +0x004 DeviceObject   : 0x9f509648 _DEVICE_OBJECT
   +0x008 Flags          : 0x412
   +0x00c DriverStart    : 0x8c300000 Void
   +0x010 DriverSize     : 0xe000
   +0x014 DriverSection  : 0x8a556ba8 Void
   +0x018 DriverExtension  : 0x8a5575c8 _DRIVER_EXTENSION
   +0x01c DriverName     : _UNICODE_STRING "\Driver\kbdclass"
   +0x024 HardwareDatabase : 0x815c2c28 _UNICODE_STRING
"\REGISTRY\MACHINE\HARDWARE\
DESCRIPTION\SYSTEM"
   +0x028 FastIoDispatch   : (null)
   +0x02c DriverInit     : 0x8c30a010     long  +ffffffff8c30a010
   +0x030 DriverStartIo    : (null)
   +0x034 DriverUnload     : (null)
   +0x038 MajorFunction    : [28] 0x8c301d80     long  +ffffffff8c301d80
1: kd> dt nt!_device_object 9f509648
   +0x000 Type           : 0n3
   +0x002 Size           : 0x1a8
   +0x004 ReferenceCount   : 0n0
   +0x008 DriverObject     : 0x8a557520 _DRIVER_OBJECT
```

```
+0x00c NextDevice       : 0x8bf2fe20 _DEVICE_OBJECT
+0x010 AttachedDevice   : (null)
+0x014 CurrentIrp       : (null)
+0x018 Timer            : (null)
+0x01c Flags            : 0x2044
+0x020 Characteristics  : 0x100
+0x024 Vpb              : (null)
+0x028 DeviceExtension  : 0x9f509700 Void
+0x02c DeviceType       : 0xb
+0x030 StackSize        : 7 ''
+0x034 Queue            : <unnamed-tag>
+0x05c AlignmentRequirement : 0
+0x060 DeviceQueue      : _KDEVICE_QUEUE
+0x074 Dpc              : _KDPC
+0x094 ActiveThreadCount : 0
+0x098 SecurityDescriptor : 0x82090930 Void
...
```

There are some interesting fields in these structures, which we'll discuss in the next section.

Using objects to record information about drivers means that the I/O manager doesn't need to know details about individual drivers. The I/O manager merely follows a pointer to locate a driver, thereby providing a layer of portability and allowing new drivers to be loaded easily.

Opening devices

A *file object* is a kernel-mode data structure that represents a handle to a device. File objects clearly fit the criteria for objects in Windows: They are system resources that two or more user-mode processes can share; they can have names; they are protected by object-based security; and they support synchronization. Shared resources in the I/O system, like those in other components of the Windows executive, are manipulated as objects. (See Chapter 8 in Part 2 for more on object management.)

File objects provide a memory-based representation of resources that conform to an I/O-centric interface, in which they can be read from or written to. **Table 6-1** lists some of the file object's attributes. For specific field declarations and sizes, see the structure definition for `FILE_OBJECT` in wdm.h.

Attribute	Purpose
File name	This identifies the virtual file that the file object refers to, which was passed in to the `CreateFile` or `CreateFile2` APIs.
Current byte offset	This identifies the current location in the file (valid only for synchronous I/O).
Share modes	These indicate whether other callers can open the file for read, write, or delete operations while the current caller is using it.
Open mode flags	These indicate whether I/O will be synchronous or asynchronous, cached or non-cached, sequential or random, and so on.
Pointer to device object	This indicates the type of device the file resides on.
Pointer to the volume parameter block (VPB)	This indicates the volume, or partition, that the file resides on (in the case of file system files).
Pointer to section object pointers	This indicates a root structure that describes a mapped/cached file. This structure also contains the shared cache map, which identifies which parts of the file are cached (or rather, mapped) by the cache manager and where they reside in the cache.
Pointer to private cache map	This is used to store per-handle caching information such as the read patterns for this handle or the page priority for the process. See Chapter 5, "Memory management," for more information on page priority.
List of I/O request packets (IRPs)	If thread-agnostic I/O (described in the section "Thread-agnostic I/O" later in this chapter) is used and the file object is associated with a completion port (described in the section "I/O completion ports"), this is a list of all the I/O operations that are associated with this file object.
I/O completion context	This is context information for the current I/O completion port, if one is active.
File object extension	This stores the I/O priority (explained later in this chapter) for the file and whether share-access checks should be performed on the file object, and contains optional file object extensions that store context-specific information.

TABLE 6-1 File object attributes

To maintain some level of opacity toward driver code that uses the file object, and to enable extending the file object functionality without enlarging the structure, the file object also contains an extension field, which allows for up to six different kinds of additional attributes, described in **Table 6-2**.

Extension	Purpose
Transaction parameters	This contains the transaction parameter block, which contains information about a transacted file operation. It's returned by `IoGetTransactionParameterBlock`.
Device object hint	This identifies the device object of the filter driver with which this file should be associated. It's set with `IoCreateFileEx` or `IoCreateFileSpecifyDeviceObjectHint`.
I/O status block range	This allows applications to lock a user-mode buffer into kernel-mode memory to optimize asynchronous I/Os. It's set with `SetFileIoOverlappedRange`.
Generic	This contains filter driver–specific information, as well as extended create parameters (ECPs) that were added by the caller. It's set with `IoCreateFileEx`.
Scheduled file I/O	This stores a file's bandwidth reservation information, which is used by the storage system to optimize and guarantee throughput for multimedia applications. (See the section "Bandwidth reservation (scheduled file I/O)" later in this chapter.) It's set with `SetFileBandwidthReservation`.
Symbolic link	This is added to the file object upon creation, when a mount point or directory junction is traversed (or a filter explicitly reparses the path). It stores the caller-supplied path, including information about any intermediate junctions, so that if a relative symbolic link is hit, it can walk back through the junctions. See Chapter 13 in Part 2 for more information on NTFS symbolic links, mount points, and directory junctions.

TABLE 6-2 File object extensions

When a caller opens a file or a simple device, the I/O manager returns a handle to a file object. Before that happens, the driver responsible for the device in question is asked via its Create dispatch routine (`IRP_MJ_CREATE`) whether it's OK to open the device and allow the driver to perform any initialization necessary if the open request is to succeed.

NOTE

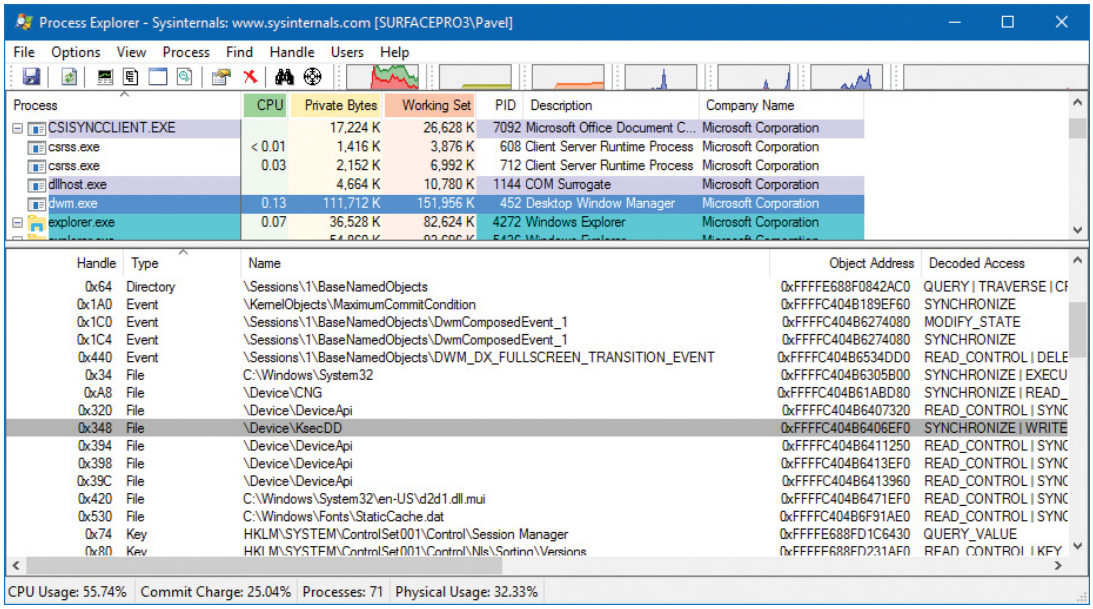
File objects represent open instances of files, not files themselves. Unlike UNIX systems, which use vnodes, Windows does not define the representation of a file; Windows file-system drivers define their own representations.

Similar to executive objects, files are protected by a security descriptor that contains an access control list (ACL). The I/O manager consults the security subsystem to determine whether a file's ACL allows the process to access the file in the way its thread is requesting. If it does, the object manager grants the access and associates the granted access rights with the file handle that it returns. If this thread or another thread in the process needs to perform additional operations not specified in the original request, the thread must open the same file again with a different request (or duplicate the handle with the requested access) to get

another handle, which prompts another security check. (See **Chapter 7** for more information about object protection.)

EXPERIMENT: VIEWING DEVICE HANDLES

Any process that has an open handle to a device will have a file object in its handle table corresponding to the open instance. You can view these handles with Process Explorer by selecting a process and checking **Handles** in the **Lower Pane View** submenu of the **View** menu. Sort by the **Type** column and scroll to where you see the handles that represent file objects, which are labeled as *File*.



In this example, the Desktop Windows Manager (dwm.exe) process has a handle open to a device created by the kernel security device driver (Ksecdd.sys). You can look at the specific file object in the kernel debugger by first identifying the address of the object. The following command reports information on the highlighted handle (handle value 0x348) in the preceding screenshot, which is in the Dwm.exe process that has a process ID of 452 decimal:

Click here to view code image

```
lkd> !handle 348 f 0n452

PROCESS ffffc404b62fb780
    SessionId: 1  Cid: 01c4    Peb: b4c3db0000  ParentCid: 0364
    DirBase: 7e607000  ObjectTable: ffffe688fd1c38c0  HandleCount:
<Data Not Accessible>
    Image: dwm.exe
```

Handle Error reading handle count.

0348: Object: **`ffffc404b6406ef0`** GrantedAccess: 00100003 (Audit)
Entry: ffffe688fd396d20
Object: ffffc404b6406ef0 Type: (ffffc404b189bf20) File
ObjectHeader: ffffc404b6406ec0 (new version)
HandleCount: 1 PointerCount: 32767

Because the object is a file object, you can get information about it with the `!fileobj` command (notice it's also the same object address shown in Process Explorer):

Click here to view code image

lkd> !fileobj **`ffffc404b6406ef0`**

Device Object: 0xffffc404b2fa7230 \Driver\KSecDD
Vpb is NULL
Event signalled

Flags: 0x40002
Synchronous IO
Handle Created

CurrentByteOffset: 0

Because a file object is a memory-based representation of a shareable resource and not the resource itself, it's different from other executive objects. A file object contains only data that is unique to an object handle, whereas the file itself contains the data or text to be shared. Each time a thread opens a file, a new file object is created with a new set of handle-specific attributes. For example, for files opened synchronously, the current byte offset attribute refers to the location in the file at which the next read or write operation using that handle will occur. Each handle to a file has a private byte offset even though the underly-

ing file is shared. A file object is also unique to a process—except when a process duplicates a file handle to another process (by using the Windows `DuplicateHandle` function) or when a child process inherits a file handle from a parent process. In these situations, the two processes have separate handles that refer to the same file object.

Although a file handle is unique to a process, the underlying physical resource is not. Therefore, as with any shared resource, threads must synchronize their access to shareable resources such as files, file directories, and devices. If a thread is writing to a file, for example, it should specify exclusive write access when opening the file to prevent other threads from writing to the file at the same time. Alternatively, by using the Windows `LockFile` function, the thread could lock a portion of the file while writing to it when exclusive access is required.

When a file is opened, the file name includes the name of the device object on which the file resides. For example, the name \Device\HarddiskVolume1\Myfile.dat may refer to the file Myfile.dat on the C: volume. The substring \Device\HarddiskVolume1 is the name of the internal Windows device object representing that volume. When opening Myfile.dat, the I/O manager creates a file object and stores a pointer to the HarddiskVolume1 device object in the file object and then returns a file handle to the caller. Thereafter, when the caller uses the file handle, the I/O manager can find the HarddiskVolume1 device object directly.

Keep in mind that internal Windows device names can't be used in Windows applications—instead, the device name must appear in a special directory in the object manager's namespace, which is \GLOBAL??. This directory contains symbolic links to the real, internal Windows device names. As was described earlier, device drivers are responsible for creating links in this directory so that their devices will be accessible to Windows applications. You can examine or even change these links programmatically with the Windows `QueryDosDevice` and `DefineDosDevice` functions.

I/O processing

Now that we've covered the structure and types of drivers and the data structures that support them, let's look at how I/O requests flow through the system. I/O requests pass through several predictable stages of processing. The stages vary depending on whether the request is destined for a device operated by a single-layered driver or for a device reached through a multilayered driver. Processing varies further depending on whether the caller specified synchronous or asynchronous I/O, so we'll begin our discussion of I/O types with these two and then move on to others.

Types of I/O

Applications have several options for the I/O requests they issue. Furthermore, the I/O manager gives drivers the choice of implementing a shortcut I/O interface that can often mitigate IRP allocation for I/O processing. In this section, we'll explain these options for I/O requests.

Synchronous and asynchronous I/O

Most I/O operations issued by applications are *synchronous* (which is the default). That is, the application thread waits while the device performs the data operation and returns a status code when the I/O is complete. The program can then continue and access the transferred data immediately. When used in their simplest form, the Windows `ReadFile` and `WriteFile` functions are executed synchronously. They complete the I/O operation before returning control to the caller.

Asynchronous I/O allows an application to issue multiple I/O requests and continue executing while the device performs the I/O operation. This type of I/O can improve an application's throughput because it allows the application thread to continue with other work while an I/O operation is in progress. To use asynchronous I/O, you must specify the `FILE_FLAG_OVERLAPPED` flag when you call the Windows `CreateFile` or `CreateFile2` functions. Of course, after issuing an asynchronous I/O operation, the thread must be careful not to access any data from the

I/O operation until the device driver has finished the data operation. The thread must synchronize its execution with the completion of the I/O request by monitoring a handle of a synchronization object (whether that's an event object, an I/O completion port, or the file object itself) that will be signaled when the I/O is complete.

Regardless of the type of I/O request, I/O operations issued to a driver on behalf of the application are performed asynchronously. That is, once an I/O request has been initiated, the device driver must return to the I/O system as soon as possible. Whether or not the I/O system returns immediately to the caller depends on whether the handle was opened for synchronous or asynchronous I/O. **Figure 6-3** illustrates the flow of control when a read operation is initiated. Notice that if a wait is done, which depends on the overlapped flag in the file object, it is done in kernel mode by the `NtReadFile` function.

You can test the status of a pending asynchronous I/O operation with the Windows `HasOverlapped- IoCompleted` macro or get more details with the `GetOverlappedResult(Ex)` functions. If you're using I/O completion ports (described in the "**I/O completion ports**" section later in this chapter), you can use the `GetQueuedCompletionStatus(Ex)` function(s).

Fast I/O

Fast I/O is a special mechanism that allows the I/O system to bypass the generation of an IRP and instead go directly to the driver stack to complete an I/O request. This mechanism is used for optimizing certain I/O paths, which are somewhat slower when using IRPs. (Fast I/O is described in detail in Chapter 13 and Chapter 14 in Part 2.) A driver registers its fast I/O entry points by entering them in a structure pointed to by the `PFAST_IO_DISPATCH` pointer in its driver object.

EXPERIMENT: LOOKING AT A DRIVER'S REGISTERED FAST I/O ROUTINES

The `!drvobj` kernel debugger command can list the fast I/O routines that a driver registers in its driver object. Typically, however, only file-system drivers have any use for fast I/O routines—although there are exceptions, such as network protocol drivers and bus filter drivers. The following output shows the fast I/O table for the NTFS file-system driver object:

Click here to view code image

```
lkd> !drvobj \filesystem\ntfs 2
Driver object (ffffc404b2fbf810) is for:
 \FileSystem\NTFS
DriverEntry:   fffff80e5663a030                NTFS!GsDriverEntry
DriverStartIo: 00000000
DriverUnload:  00000000
AddDevice:     00000000

Dispatch routines:
...
Fast I/O routines:
FastIoCheckIfPossible            fffff80e565d6750
NTFS!NtfsFastIoCheckIfPossible
FastIoRead                       fffff80e56526430  NTFS!NtfsCopyReadA
FastIoWrite                      fffff80e56523310  NTFS!NtfsCopyWriteA
FastIoQueryBasicInfo             fffff80e56523140
NTFS!NtfsFastQueryBasicInfo
FastIoQueryStandardInfo          fffff80e56534d20  NTFS!NtfsFastQueryStdInfo
FastIoLock                       fffff80e5651e610  NTFS!NtfsFastLock
FastIoUnlockSingle               fffff80e5651e3c0  NTFS!NtfsFastUnlockSingle
FastIoUnlockAll                  fffff80e565d59e0  NTFS!NtfsFastUnlockAll
FastIoUnlockAllByKey             fffff80e565d5c50
NTFS!NtfsFastUnlockAllByKey
ReleaseFileForNtCreateSection    fffff80e5644fd90  NTFS!NtfsReleaseForCreate
Section
FastIoQueryNetworkOpenInfo       fffff80e56537750  NTFS!NtfsFastQueryNetwork
```

```
OpenInfo
AcquireForModWrite              fffff80e5643e0c0
NTFS!NtfsAcquireFileForModWrite
MdlRead                 fffff80e5651e950    NTFS!NtfsMdlReadA
MdlReadComplete             fffff802dc6cd844
nt!FsRtlMdlReadCompleteDev
PrepareMdlWrite             fffff80e56541a10    NTFS!NtfsPrepareMdlWriteA
MdlWriteComplete            fffff802dcb76e48
nt!FsRtlMdlWriteCompleteDev
FastIoQueryOpen             fffff80e5653a520
NTFS!NtfsNetworkOpenCreate
ReleaseForModWrite              fffff80e5643e2c0
NTFS!NtfsReleaseFileForModWrite
AcquireForCcFlush               fffff80e5644ca60
NTFS!NtfsAcquireFileForCcFlush
ReleaseForCcFlush               fffff80e56450cf0
NTFS!NtfsReleaseFileForCcFlush
```

The output shows that NTFS has registered its `NtfsCopyReadA` routine as the fast I/O table's `FastIoRead` entry. As the name of this fast I/O entry implies, the I/O manager calls this function when issuing a read I/O request if the file is cached. If the call doesn't succeed, the standard IRP path is selected.

Mapped-file I/O and file caching

Mapped-file I/O is an important feature of the I/O system—one that the I/O system and the memory manager produce jointly. (See **Chapter 5** for details on how mapped files are implemented.) *Mapped-file I/O* refers to the ability to view a file residing on disk as part of a process's virtual memory. A program can access the file as a large array without buffering data or performing disk I/O. The program accesses memory, and the memory manager uses its paging mechanism to load the correct page from the disk file. If the application writes to its virtual ad-

dress space, the memory manager writes the changes back to the file as part of normal paging.

Mapped-file I/O is available in user mode through the Windows `CreateFileMapping`, `MapViewOfFile`, and related functions. Within the operating system, mapped-file I/O is used for important operations such as file caching and image activation (loading and running executable programs). The other major consumer of mapped-file I/O is the cache manager. File systems use the cache manager to map file data in virtual memory to provide better response time for I/O-bound programs. As the caller uses the file, the memory manager brings accessed pages into memory. Whereas most caching systems allocate a fixed number of bytes for caching files in memory, the Windows cache grows or shrinks depending on how much memory is available. This size variability is possible because the cache manager relies on the memory manager to automatically expand (or shrink) the size of the cache using the normal working set mechanisms explained in **Chapter 5**—in this case applied to the system working set. By taking advantage of the memory manager's paging system, the cache manager avoids duplicating the work that the memory manager already performs. (The workings of the cache manager are explained in detail in Chapter 14 in Part 2.)

Scatter/gather I/O

Windows supports a special kind of high-performance I/O called *scatter/gather*, available via the Windows `ReadFileScatter` and `WriteFileGather` functions. These functions allow an application to issue a single read or write from more than one buffer in virtual memory to a contiguous area of a file on disk instead of issuing a separate I/O request for each buffer. To use scatter/gather I/O, the file must be opened for non-cached I/O, the user buffers being used must be page-aligned, and the I/Os must be asynchronous (overlapped). Furthermore, if the I/O is directed at a mass storage device, the I/O must be aligned on a device sector boundary and have a length that is a multiple of the sector size.

I/O request packets

An *I/O request packet* (*IRP*) is where the I/O system stores information it needs to process an I/O request. When a thread calls an I/O API, the I/O manager constructs an IRP to represent the operation as it progresses through the I/O system. If possible, the I/O manager allocates IRPs from one of three per-processor IRP non-paged look-aside lists:

- **The small-IRP look-aside list** This stores IRPs with one stack location. (IRP stack locations are described shortly.)

- **The medium-IRP look-aside list** This contains IRPs with four stack locations (which can also be used for IRPs that require only two or three stack locations).

- **The large-IRP look-aside list** This contains IRPs with more than four stack locations. By default, the system stores IRPs with 14 stack locations on the large-IRP look-aside list, but once per minute, the system adjusts the number of stack locations allocated and can increase it up to a maximum of 20, based on how many stack locations have been recently required.

These lists are also backed by global look-aside lists as well, allowing efficient cross-CPU IRP flow. If an IRP requires more stack locations than are contained in the IRPs on the large-IRP look-aside list, the I/O manager allocates IRPs from non-paged pool. The I/O manager allocates IRPs with the `IoAllocate-Irp` function, which is also available for device-driver developers, because in some cases a driver may want to initiate an I/O request directly by creating and initializing its own IRPs. After allocating and initializing an IRP, the I/O manager stores a pointer to the caller's file object in the IRP.

NOTE

If defined, the DWORD registry value `LargeIrpStackLocations` in the HKLM\System\CurrentControlSet\Session Manager\I/O System key specifies how many stack locations are contained in IRPs stored on the large-IRP look-aside list. Similarly, the `MediumIrpStackLocations` value in the same key can be used to change the size of IRP stack locations on the medium-IRP look-aside list.

Figure 6-11 shows some of the important members of the IRP structure. It is always accompanied by one or more `IO_STACK_LOCATION` objects (described in the next section).

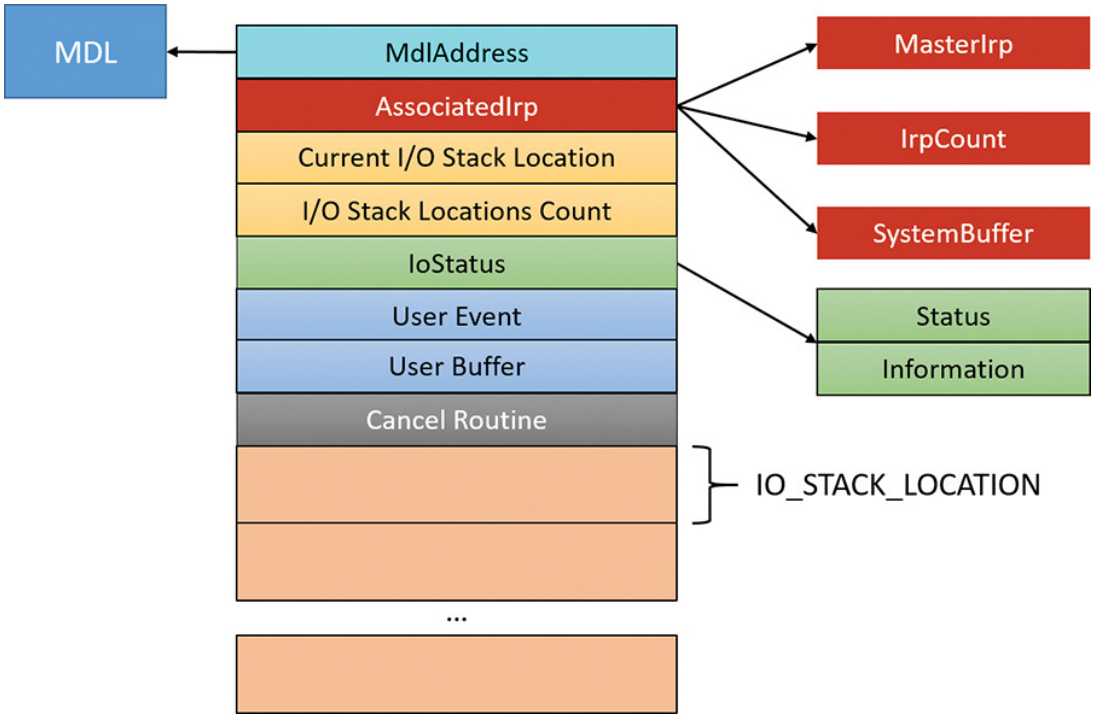


FIGURE 6-11 Important members of the IRP structure.

Here is a quick rundown of the members:

■ **IoStatus** This is the status of the IRP, consisting of two members; `Status`, which is the actual code itself and `Information`, a polymorphic value that has meaning in some cases. For example, for a read or write operation, this value (set by the driver) indicates the number of

bytes read or written. This same value is the one reported as an output value from the functions `ReadFile` and `WriteFile`.

■ **MdlAddress** This is an optional pointer to a memory descriptor list (MDL). An MDL is a structure that represents information for a buffer in physical memory. We'll discuss its main usage in device drivers in the next section. If an MDL was not requested, the value is `NULL`.

■ **I/O stack locations count and current stack location** These store the total number of trailing I/O stack location objects and point to the current one that this driver layer should look at, respectively. The next section discusses I/O stack locations in detail.

■ **User buffer** This is the pointer to the buffer provided by the client that initiated the I/O operation. For example, it is the buffer provided to the `ReadFile` or `WriteFile` functions.

■ **User event** This is the kernel event object that was used with an overlapped (asynchronous) I/O operation (if any). An event is one way to be notified when the I/O operation completes.

■ **Cancel routine** This is the function to be called by the I/O manager in case the IRP is cancelled.

■ **AssociatedIrp** This is a union of one of three fields. The `SystemBuffer` member is used in case the I/O manager used the buffered I/O technique for passing the user's buffer to the driver. The next section discusses buffered I/O, as well as other options for passing user mode buffers to drivers. The `MasterIrp` member provides a way to create a "master IRP" that splits its work into sub-IRPs, where the master is considered complete only when all its sub-IRPs have completed.

I/O stack locations

An IRP is always followed by one or more I/O stack locations. The number of stack locations is equal to the number of layered devices in the device node the IRP is destined for. The I/O operation information is split between the IRP body (the main structure) and the current I/O stack location, where *current* means the one set up for the particular layer of devices. **Figure 6-12** shows the important fields of an I/O stack location. When an IRP is created, the number of requested I/O stack locations is passed to `IoAllocateIrp`. The I/O manager then initializes the IRP body and the first I/O stack location only, destined for the topmost device in the device node. Each layer in the device node is responsible for initializing the next I/O stack location if it decides to pass the IRP down to the next device.

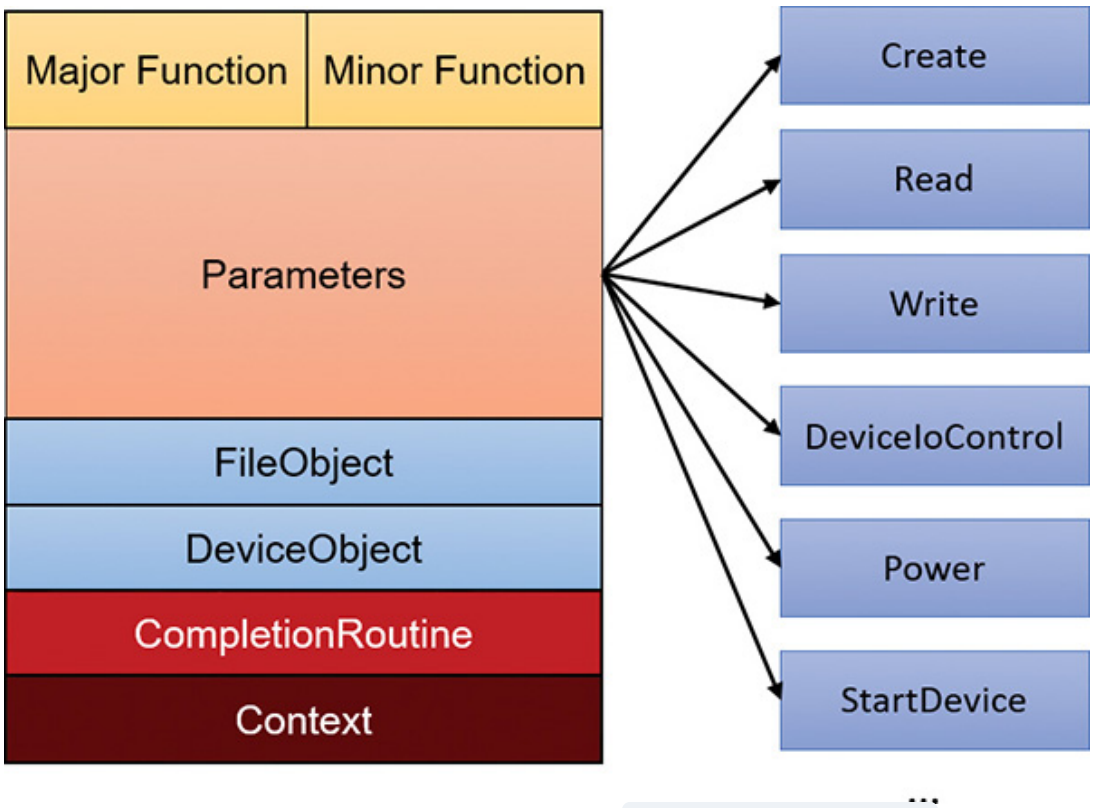


FIGURE 6-12 Important members of the `IO_STACK_LOCATION` structure.

Here is a rundown of the members shown in **Figure 6-12**:

■ **Major function** This is the primary code that indicates the type of request (read, write, create, Plug and Play, and so on), also known as dispatch routine code. It's one of 28 constants (0 to 27) starting with

`IRP_MJ_` in wdm.h. This index is used by the I/O manager into the `MajorFunction` array of function pointers in the driver object to jump to the appropriate routine within a driver. Most drivers specify dispatch routines to handle only a subset of possible major function codes, including create (open), read, write, device I/O control, power, Plug and Play, system control (for WMI commands), cleanup, and close. File-system drivers are an example of a driver type that often fills in most or all of its dispatch entry points with functions. In contrast, a driver for a simple USB device would probably fill in only the routines needed for open, close, read, write, and sending I/O control codes. The I/O manager sets any dispatch entry points that a driver doesn't fill to point to its own `IopInvalidDeviceRequest`, which completes the IRP with an error status indicating that the major function specified in the IRP is invalid for that device.

- **Minor function** This is used to augment the major function code for some functions. For example, `IRP_MJ_READ` (read) and `IRP_MJ_WRITE` (write) have no minor functions. But Plug and Play and Power IRPs always have a minor IRP code that specializes the general major code. For example, the Plug and Play `IRP_MJ_PNP` major code is too generic; the exact instruction is given by the minor IRP, such as `IRP_MN_START_DEVICE`, `IRP_MN_REMOVE_DEVICE`, and so on.

- **Parameters** This is a monstrous union of structures, each of which valid for a particular major function code or a combination of major/minor codes. For example, for a read operation (`IRP_MJ_READ`), the `Parameters.Read` structure holds information on the read request, such as the buffer size.

- **File object and Device object** These point to the associated `FILE_OBJECT` and `DEVICE_OBJECT` for this I/O request.

- **Completion routine** This is an optional function that a driver can register with the `IoSetCompletionRoutine(Ex)` DDI, to be called when the IRP is completed by a lower layer driver. At that point, the driver can look at the completion status of the IRP and do any needed post-

processing. It can even undo the completion (by returning the special value `STATUS_MORE_PROCESSING_REQUIRED` from the function) and re-send the IRP (perhaps with modified parameters) to the device node—or even a different device node—again.

■ **Context** This is an arbitrary value set with the `IoSetCompletionRoutine(Ex)` call that is passed, as is, to the completion routine.

The split of information between the IRP body and its I/O stack location allows for the changing of I/O stack location parameters for the next device in the device stack, while keeping the original request parameters. For example, a read IRP targeted at a USB device is often changed by the function driver to a device I/O control IRP where the input buffer argument of the device control points to a USB request packet (URB) that is understood by the lower-layer USB bus driver. Also, note that completion routines can be registered by any layer (except the bottom-most one), each having its own place in an I/O stack location (the completion routine is stored in the *next* lower I/O stack location).

EXPERIMENT: LOOKING AT DRIVER DISPATCH ROUTINES

You can obtain a list of the functions a driver has defined for its dispatch routines by using bit 1 (value of 2) with the `!drvobj` kernel debugger command. The following output shows the major function codes supported by the NTFS driver. (This is the same experiment as with fast I/O.)

Click here to view code image

```
lkd> !drvobj \filesystem\ntfs 2
Driver object (ffffc404b2fbf810) is for:
 \FileSystem\NTFS
DriverEntry:   fffff80e5663a030                NTFS!GsDriverEntry
DriverStartIo: 00000000
DriverUnload:  00000000
AddDevice:     00000000

Dispatch routines:
[00] IRP_MJ_CREATE                 fffff80e565278e0    NTFS!NtfsFsdCreate
[01]
IRP_MJ_CREATE_NAMED_PIPE       fffff802dc762c80    nt!IopInvalidDeviceRequest
[02] IRP_MJ_CLOSE                  fffff80e565258c0    NTFS!NtfsFsdClose
[03] IRP_MJ_READ                   fffff80e56436060    NTFS!NtfsFsdRead
[04] IRP_MJ_WRITE                  fffff80e564461d0    NTFS!NtfsFsdWrite
[05]
IRP_MJ_QUERY_INFORMATION       fffff80e565275f0    NTFS!NtfsFsdDispatchWait
[06]
IRP_MJ_SET_INFORMATION         fffff80e564edb80    NTFS!NtfsFsdSetInformation
[07]
IRP_MJ_QUERY_EA                fffff80e565275f0    NTFS!NtfsFsdDispatchWait
[08]
IRP_MJ_SET_EA                  fffff80e565275f0    NTFS!NtfsFsdDispatchWait
[09]
IRP_MJ_FLUSH_BUFFERS           fffff80e5653c9a0    NTFS!NtfsFsdFlushBuffers
[0a]
IRP_MJ_QUERY_VOLUME_INFORMATION  fffff80e56538d10    NTFS!NtfsFsdDispatch
```

[0b]
IRP_MJ_SET_VOLUME_INFORMATION fffff80e56538d10 NTFS!NtfsFsdDispatch
[0c] IRP_MJ_DIRECTORY_CONTROL fffff80e564d7080
NTFS!NtfsFsdDirectoryControl
[0d] IRP_MJ_FILE_SYSTEM_CONTROL fffff80e56524b20
NTFS!NtfsFsdFileSystemControl
[0e]
IRP_MJ_DEVICE_CONTROL fffff80e564f9de0 NTFS!NtfsFsdDeviceControl
[0f]
IRP_MJ_INTERNAL_DEVICE_CONTROL fffff802dc762c80 nt!IopInvalidDeviceRequest
[10]
IRP_MJ_SHUTDOWN fffff80e565efb50 NTFS!NtfsFsdShutdown
[11]
IRP_MJ_LOCK_CONTROL fffff80e5646c870 NTFS!NtfsFsdLockControl
[12]
IRP_MJ_CLEANUP fffff80e56525580 NTFS!NtfsFsdCleanup
[13]
IRP_MJ_CREATE_MAILSLOT fffff802dc762c80 nt!IopInvalidDeviceRequest
[14]
IRP_MJ_QUERY_SECURITY fffff80e56538d10 NTFS!NtfsFsdDispatch
[15]
IRP_MJ_SET_SECURITY fffff80e56538d10 NTFS!NtfsFsdDispatch
[16]
IRP_MJ_POWER fffff802dc762c80 nt!IopInvalidDeviceRequest
[17]
IRP_MJ_SYSTEM_CONTROL fffff802dc762c80 nt!IopInvalidDeviceRequest
[18]
IRP_MJ_DEVICE_CHANGE fffff802dc762c80 nt!IopInvalidDeviceRequest
[19]
IRP_MJ_QUERY_QUOTA fffff80e565275f0 NTFS!NtfsFsdDispatchWait
[1a]
IRP_MJ_SET_QUOTA fffff80e565275f0 NTFS!NtfsFsdDispatchWait
[1b] IRP_MJ_PNP fffff80e56566230 NTFS!NtfsFsdPnp

Fast I/O routines:

...

While active, each IRP is usually queued in an IRP list associated with the thread that requested the I/O. (Otherwise, it is stored in the file object when performing thread-agnostic I/O, which is described in the "Thread agnostic I/O" section, later in this chapter.) This allows the I/O system to find and cancel any outstanding IRPs if a thread terminates with I/O requests that have not been completed. Additionally, paging I/O IRPs are also associated with the faulting thread (although they are not cancellable). This allows Windows to use the thread-agnostic I/O optimization—when an asynchronous procedure call (APC) is not used to complete I/O if the current thread is the initiating thread. This means page faults occur inline instead of requiring APC delivery.

EXPERIMENT: LOOKING AT A THREAD'S OUTSTANDING IRPS

The `!thread` command prints any IRPs associated with the thread. The `!process` command does this as well, if requested. Run the kernel debugger with local or live debugging and list the threads of an explorer process:

Click here to view code image

```
lkd> !process 0 7 explorer.exe
PROCESS ffffc404b673c780
    SessionId: 1  Cid: 10b0    Peb: 00cbb000  ParentCid: 1038
    DirBase: 8895f000  ObjectTable: ffffe689011b71c0  HandleCount:
<Data Not
Accessible>
    Image: explorer.exe
    VadRoot ffffc404b672b980 Vads 569 Clone 0 Private 7260. Modified
366527. Locked 784.
    DeviceMap ffffe688fd7a5d30
    Token                             ffffe68900024920
    ElapsedTime                       18:48:28.375
    UserTime                          00:00:17.500
    KernelTime                        00:00:13.484
    ...
    MemoryPriority                    BACKGROUND
    BasePriority                      8
    CommitCharge                      10789
    Job                               ffffc404b6075060

        THREAD ffffc404b673a080  Cid 10b0.10b4  Teb: 0000000000cbc000
Win32Thread:
ffffc404b66e7090 WAIT: (WrUserRequest) UserMode Non-Alertable
            ffffc404b6760740  SynchronizationEvent
        Not impersonating
...

        THREAD ffffc404b613c7c0  Cid 153c.15a8  Teb: 00000000006a3000
```

```
Win32Thread:
ffffc404b6a83910 WAIT: (UserRequest) UserMode Non-Alertable
        ffffc404b58d0d60  SynchronizationEvent
        ffffc404b566f310  SynchronizationEvent
    IRP List:
        ffffc404b69ad920: (0006,02c8) Flags: 00060800  Mdl: 00000000
...
```

You should see many threads, with most of them having IRPs reported in the `IRP List` section of the thread information (note that the debugger will show only the first 17 IRPs for a thread that has more than 17 outstanding I/O requests). Choose an IRP and examine it with the `!irp` command:

Click here to view code image

```
lkd> !irp ffffc404b69ad920
Irp is active with 2 stacks 1 is current (= 0xffffc404b69ad9f0)
 No Mdl: No System Buffer: Thread ffffc404b613c7c0:  Irp stack trace.
    cmd  flg cl Device   File     Completion-Context
>[IRP_MJ_FILE_SYSTEM_CONTROL(d), N/A(0)]
        5 e1 ffffc404b253cc90 ffffc404b5685620 fffff80e55752ed0-
ffffc404b63c0e00
Success Error Cancel pending
            \FileSystem\Npfs      FLTMGR!FltpPassThroughCompletion
                Args: 00000000 00000000 00110008 00000000
 [IRP_MJ_FILE_SYSTEM_CONTROL(d), N/A(0)]
        5  0 ffffc404b3cdca00 ffffc404b5685620 00000000-00000000
            \FileSystem\FltMgr
                Args: 00000000 00000000 00110008 00000000
```

The IRP has two stack locations and is targeted at a device owned by the Named Pipe File System (NPFS) driver. (NPFS is described in Chapter 10, “Networking,” in Part 2.)

IRP flow

IRPs are typically created by the I/O manager, and then sent to the first device on the target device node. **Figure 6-13** shows a typical IRP flow for hardware-based device drivers.

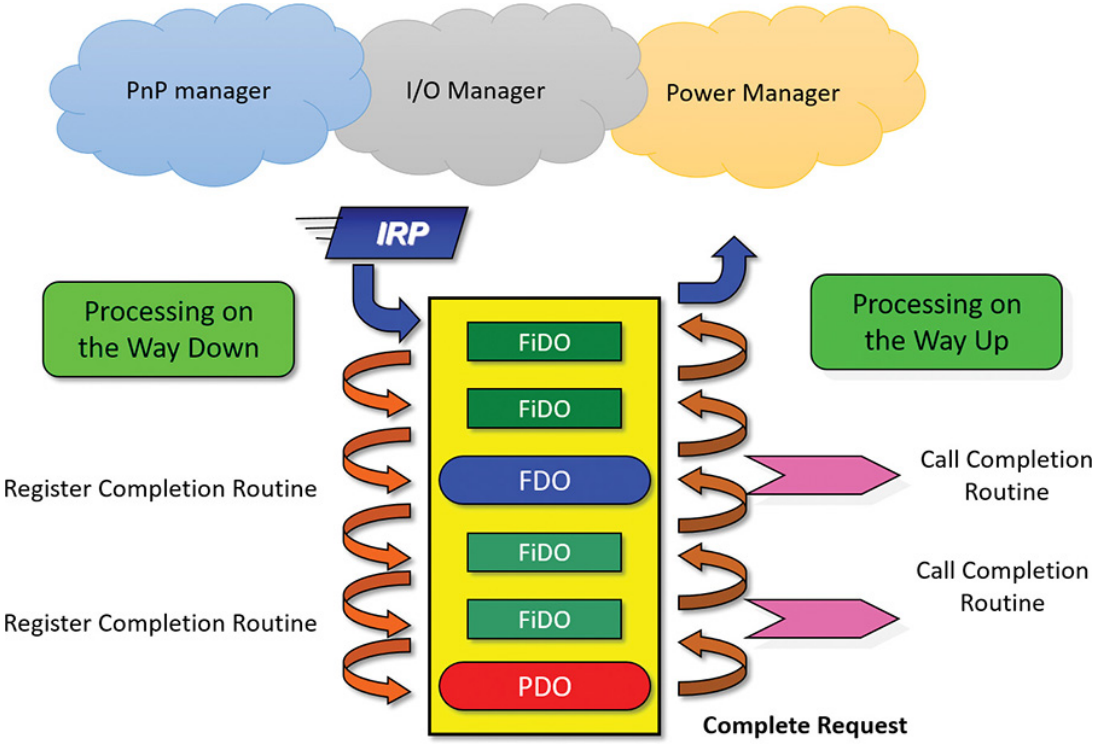


FIGURE 6-13 IRP flow.

The I/O manager is not the only entity that creates IRPs. The Plug and Play manager and the Power manager are also responsible for creating IRPs with major function code `IRP_MJ_PNP` and `IRP_MJ_POWER`, respectively.

Figure 6-13 shows an example device node with six layered device objects: two upper filters, the FDO, two lower filters, and the PDO. This means an IRP targeted at this devnode is created with six I/O stack locations—one for each layer. An IRP is always delivered to the highest layered device, even if a handle was opened to a named device that is lower in the device stack.

A driver that receives an IRP can do one of the following:

■ It can complete the IRP then and there by calling `IoCompleteRequest`. This could be because the IRP has some invalid parameters (for example, insufficient buffer size or bad I/O control code), or because the operation requested is quick and can be accom-

plished immediately, such as getting some status from the device or reading a value from the registry. The driver calls `IoGetCurrentIrpStackLocation` to get a pointer to the stack location that it should refer to.

- The driver can forward the IRP to the next layer after optionally doing some processing. For example, an upper filter can do some logging of the operation and send the IRP down to be executed normally. Before sending the request down, the driver must prepare the next I/O stack location that would be looked at by the next driver in line. It can use the `IoSkipCurrentIrpStackLocation` macro if it does not wish to make changes, or it can make a copy with `IoCopyIrpStackLocationToNext` and make changes to the copied stack location by getting a pointer with `IoGetNextIrpStackLocation` and making appropriate changes. Once the next I/O stack location is prepared, the driver calls `IoCallDriver` to do the actual IRP forwarding.

- As an extension of the previous point, the driver can also register for a completion routine by calling `IoSetCompletionRoutine(Ex)` before passing down the IRP. Any layer except the bottom-most one can register a completion routine (there is no point in registering for the bottom-most layer since that driver must complete the IRP, so no callback is needed). After `IoCompleteRequest` is called by a lower-layer driver, the IRP travels up (refer to **Figure 6-13**), calling any completion routines on the way up in reverse order of registration. In fact, the IRP originator (I/O manager, PnP manager, or power manager) use this mechanism to do any post-IRP processing and finally free the IRP.

NOTE

Because the number of devices on a given stack is known in advance, the I/O manager allocates one stack location per device driver on the stack. However, there are situations in which an IRP might be directed into a new driver stack. This can happen in scenarios involving the filter manager, which allows one filter to redirect an IRP to another filter (for example, going from a local file system to a network file system). The I/O manager exposes an API, `IoAdjustStackSizeForRedirection`, that enables this functionality by adding the required stack locations because of devices present on the redirected stack.

EXPERIMENT: VIEWING A DEVICE STACK

The `!devstack` kernel debugger command shows you the device stack of layered device objects associated with a specified device object. This example shows the device stack associated with a device object, \device\keyboardclass0, which is owned by the keyboard class driver:

Click here to view code image

```
lkd> !devstack keyboardclass0
  !DevObj         !DrvObj         !DevExt         ObjectName
> ffff9c80c0424440  \Driver\kbdclass   ffff9c80c0424590  KeyboardClass0
  ffff9c80c04247c0  \Driver\kbdhid     ffff9c80c0424910
  ffff9c80c0414060  \Driver\mshidkmdf  ffff9c80c04141b0  0000003f
!DevNode ffff9c80c0414d30 :
  DeviceInst is "HID\MSHW0029&Col01\5&1599b1c7&0&0000"
  ServiceName is "kbdhid"
```

The output highlights the entry associated with `KeyboardClass0` with the `>` character in the first column. The entries above that line are

drivers layered above the keyboard class driver, and those below are layered beneath it.

EXPERIMENT: EXAMINING IRPS

In this experiment, you'll find an uncompleted IRP on the system, and will determine the IRP type, the device at which it's directed, the driver that manages the device, the thread that issued the IRP, and what process the thread belongs to. This experiment is best performed on a 32-bit system with non-local kernel debugging. It will work with local kernel debugging as well, but IRPs may complete during the period between when commands are issued, so some instability of data should be expected.

At any point in time, there are at least a few uncompleted IRPs on a system. This occurs because there are many devices to which applications can issue IRPs that a driver will complete only when a particular event occurs, such as data becoming available. One example is a blocking read from a network endpoint. You can see the outstanding IRPs on a system with the `!irpfind` kernel debugger command (this may take some time; you can stop after some IRPs appear):

Click here to view code image

```
kd> !irpfind
Scanning large pool allocation table for tag 0x3f707249 (Irp?) (a5000000
: a5200000)

  Irp    [ Thread ] irpStack: (Mj,Mn)   DevObj  [Driver]        MDL Process
9515ad68 [aa0c04c0] irpStack: ( e, 5)  8bcb2ca0 [ \Driver\AFD]
0xaa1a3540
8bd5c548 [91deeb80] irpStack: ( e,20)  8bcb2ca0 [ \Driver\AFD]
0x91da5c40

Searching nonpaged pool (80000000 : ffc00000) for tag 0x3f707249
(Irp?)
```

86264a20 [86262040] irpStack: (e, 0) 8a7b4ef0 [\Driver\vmbus]
86278720 [91d96b80] irpStack: (e,20) 8bcb2ca0 [\Driver\AFD]
0x86270040
86279e48 [91d96b80] irpStack: (e,20) 8bcb2ca0 [\Driver\AFD]
0x86270040
862a1868 [862978c0] irpStack: (d, 0) 8bca4030 [\FileSystem\Npfs]
862a24c0 [86297040] irpStack: (d, 0) 8bca4030 [\FileSystem\Npfs]
862c3218 [9c25f740] irpStack: (c, 2) 8b127018 [\FileSystem\NTFS]
862c4988 [a14bf800] irpStack: (e, 5) 8bcb2ca0 [\Driver\AFD]
0xaa1a3540
862c57d8 [a8ef84c0] irpStack: (d, 0) 8b127018 [\FileSystem\NTFS]
0xa8e6f040
862c91c0 [99ac9040] irpStack: (3, 0) 8a7ace48 [\Driver\vmbus]
0x9517ac40
862d2d98 [9fd456c0] irpStack: (e, 5) 8bcb2ca0 [\Driver\AFD]
0x9fc11780
862d6528 [9aded800] irpStack: (c, 2) 8b127018 [\FileSystem\NTFS]
862e3230 [00000000] Irp is complete (CurrentLocation 2 > StackCount 1)
862ec248 [862e2040] irpStack: (d, 0) 8bca4030 [\FileSystem\Npfs]
862f7d70 [91dd0800] irpStack: (d, 0) 8bca4030 [\FileSystem\Npfs]
863011f8 [00000000] Irp is complete (CurrentLocation 2 > StackCount 1)
86327008 [00000000] Irp is complete (CurrentLocation 43 > StackCount
42)
86328008 [00000000] Irp is complete (CurrentLocation 43 > StackCount
42)
86328960 [00000000] Irp is complete (CurrentLocation 43 > StackCount
42)
86329008 [00000000] Irp is complete (CurrentLocation 43 > StackCount
42)
863296d8 [00000000] Irp is complete (CurrentLocation 2 > StackCount 1)
86329960 [00000000] Irp is complete (CurrentLocation 43 > StackCount
42)
89feeae0 [00000000] irpStack: (e, 0) 8a765030 [\Driver\ACPI]
8a6d85d8 [99aa1040] irpStack: (d, 0) 8b127018 [\FileSystem\NTFS]
0x00000000

8a6dc828 [8bc758c0] irpStack: (4, 0) 8b127018 [\FileSystem\NTFS] 0x00000000
8a6f42d8 [8bc728c0] irpStack: (4,34) 8b0b8030 [\Driver\disk] 0x00000000
8a6f4d28 [8632e6c0] irpStack: (4,34) 8b0b8030 [\Driver\disk] 0x00000000
8a767d98 [00000000] Irp is complete (CurrentLocation 6 > StackCount 5)
8a788d98 [00000000] irpStack: (f, 0) 00000000 [00000000: Could not read device
object or _DEVICE_OBJECT not found
]
8a7911a8 [9fdb4040] irpStack: (e, 0) 86325768 [\Driver\DeviceApi]
8b03c3f8 [00000000] Irp is complete (CurrentLocation 2 > StackCount 1)
8b0b8bc8 [863d6040] irpStack: (e, 0) 8a78f030 [\Driver\vmbus]
8b0c48c0 [91da8040] irpStack: (e, 5) 8bcb2ca0 [\Driver\AFD] 0xaa1a3540
8b118d98 [00000000] Irp is complete (CurrentLocation 9 > StackCount 8)
8b1263b8 [00000000] Irp is complete (CurrentLocation 8 > StackCount 7)
8b174008 [aa0aab80] irpStack: (4, 0) 8b127018 [\FileSystem\NTFS] 0xa15e1c40
8b194008 [aa0aab80] irpStack: (4, 0) 8b127018 [\FileSystem\NTFS] 0xa15e1c40
8b196370 [8b131880] irpStack: (e,31) 8bcb2ca0 [\Driver\AFD]
8b1a8470 [00000000] Irp is complete (CurrentLocation 2 > StackCount 1)
8b1b3510 [9fcd1040] irpStack: (e, 0) 86325768 [\Driver\DeviceApi]
8b1b35b0 [a4009b80] irpStack: (e, 0) 86325768 [\Driver\DeviceApi]
8b1cd188 [9c3be040] irpStack: (e, 0) 8bc73648 [\Driver\Beep]
...

Some IRPs are complete, and may be de-allocated very soon, or they have been de-allocated, but because the allocation from lookaside lists, the IRP has not yet been replaced with a new one.

For each IRP, its address is given, followed by the thread that issued the request. Next, the major and minor function codes for the current stack location are shown in parentheses. You can examine any IRP with the `!irp` command:

Click here to view code image

```
kd> !irp 8a6f4d28
Irp is active with 15 stacks 6 is current (= 0x8a6f4e4c)
 Mdl=8b14b250: No System Buffer: Thread 8632e6c0:  Irp stack trace.
    cmd  flg cl Device   File     Completion-Context
[N/A(0), N/A(0)]
            0  0 00000000 00000000 00000000-00000000

                        Args: 00000000 00000000 00000000 00000000
[N/A(0), N/A(0)]
            0  0 00000000 00000000 00000000-00000000

                        Args: 00000000 00000000 00000000 00000000
[N/A(0), N/A(0)]
            0  0 00000000 00000000 00000000-00000000

                        Args: 00000000 00000000 00000000 00000000
[N/A(0), N/A(0)]
            0  0 00000000 00000000 00000000-00000000

                        Args: 00000000 00000000 00000000 00000000
[N/A(0), N/A(0)]
            0  0 00000000 00000000 00000000-00000000

                        Args: 00000000 00000000 00000000 00000000
>[IRP_MJ_WRITE(4), N/A(34)]
                  14 e0 8b0b8030 00000000 876c2ef0-00000000 Success
Error Cancel
                        \Driver\disk          partmgr!PmIoCompletion
                    Args: 0004b000 00000000 4b3a0000 00000002
```

[IRP_MJ_WRITE(4), N/A(3)]
14 e0 8b0fc058 00000000 876c36a0-00000000 Success Error Cancel
\Driver\partmgr partmgr!PartitionIoCompletion
Args: 4b49ace4 00000000 4b3a0000 00000002
[IRP_MJ_WRITE(4), N/A(0)]
14 e0 8b121498 00000000 87531110-8b121a30 Success Error Cancel
\Driver\partmgr volmgr!VmpReadWriteCompletionRoutine
Args: 0004b000 00000000 2bea0000 00000002
[IRP_MJ_WRITE(4), N/A(0)]
4 e0 8b121978 00000000 82d103e0-8b1220d9 Success Error Cancel
\Driver\volmgr fvevol!FvePassThroughCompletionRdpLevel2
Args: 0004b000 00000000 4b49acdf 00000000
[IRP_MJ_WRITE(4), N/A(0)]
4 e0 8b122020 00000000 82801a40-00000000 Success Error Cancel
\Driver\fvevol rdyboost!SmdReadWriteCompletion
Args: 0004b000 00000000 2bea0000 00000002
[IRP_MJ_WRITE(4), N/A(0)]
4 e1 8b118538 00000000 828637d0-00000000 Success Error Cancel pending
\Driver\rdyboost iorate!IoRateReadWriteCompletion
Args: 0004b000 3fffffff 2bea0000 00000002
[IRP_MJ_WRITE(4), N/A(0)]
4 e0 8b11ab80 00000000 82da1610-8b1240d8 Success Error Cancel
\Driver\iorate volsnap!VspRefCountCompletionRoutine
Args: 0004b000 00000000 2bea0000 00000002
[IRP_MJ_WRITE(4), N/A(0)]
4 e1 8b124020 00000000 87886ada-89aec208 Success Error Cancel pending
\Driver\volsnap NTFS!NtfsMasterIrpSyncCompletionRoutine
Args: 0004b000 00000000 2bea0000 00000002
[IRP_MJ_WRITE(4), N/A(0)]
4 e0 8b127018 a6de4bb8 871227b2-9ef8eba8 Success Error Cancel
\FileSystem\NTFS FLTMGR!FltpPassThroughCompletion

```
                Args: 0004b000 00000000 00034000 00000000
 [IRP_MJ_WRITE(4), N/A(0)]
        4  1 8b12a3a0 a6de4bb8 00000000-00000000    pending
           \FileSystem\FltMgr
                Args: 0004b000 00000000 00034000 00000000
```

Irp Extension present at 0x8a6f4fb4:

This is a monstrous IRP with 15 stack locations (6 is current, shown in bold above, and is also specified by the debugger with the `>` character). The major and minor functions are shown for each stack location along with information on the device object and completion routines addresses.

The next step is to see what device object the IRP is targeting by executing the `!devobj` command on the device object address in the active stack location:

Click here to view code image

```
kd> !devobj 8b0b8030
Device object (8b0b8030) is for:
 DR0 \Driver\disk DriverObject 8b0a7e30
Current Irp 00000000 RefCount 1 Type 00000007 Flags 01000050
Vpb 8b0fc420 SecurityDescriptor 87da1b58 DevExt 8b0b80e8 DevObjExt
8b0b8578 Dope 8b0fc3d0
ExtensionFlags (0x00000800)  DOE_DEFAULT_SD_PRESENT
Characteristics (0x00000100)  FILE_DEVICE_SECURE_OPEN
AttachedDevice (Upper) 8b0fc058 \Driver\partmgr
AttachedTo (Lower) 8b0a4d10 \Driver\storflt
Device queue is not busy.
```

Finally, you can see details about the thread and process that issued the IRP by using the `!thread` command:

Click here to view code image

```
kd> !thread 8632e6c0
THREAD 8632e6c0  Cid 0004.0058  Teb: 00000000 Win32Thread:
00000000 WAIT:
(Executive) KernelMode Non-Alertable
   89aec20c  NotificationEvent
IRP List:
   8a6f4d28: (0006,02d4) Flags: 00060043  Mdl: 8b14b250
Not impersonating
DeviceMap                 87c025b0
Owning Process            86264280       Image:         System
Attached Process          N/A            Image:         N/A
Wait Start TickCount      8083           Ticks: 1 (0:00:00:00.015)
Context Switch Count      2223           IdealProcessor: 0
UserTime                  00:00:00.000
KernelTime                00:00:00.046
Win32 Start Address nt!ExpWorkerThread (0x81e68710)
Stack Init 89aecca0 Current 89aebeb4 Base 89aed000 Limit 89aea000
Call 00000000
Priority 13 BasePriority 13 PriorityDecrement 0 IoPriority 2
PagePriority 5
```

I/O request to a single-layered hardware-based driver

This section traces I/O requests to a single-layered kernel-mode device driver. **Figure 6-14** shows a typical IRP processing scenario for such a driver.

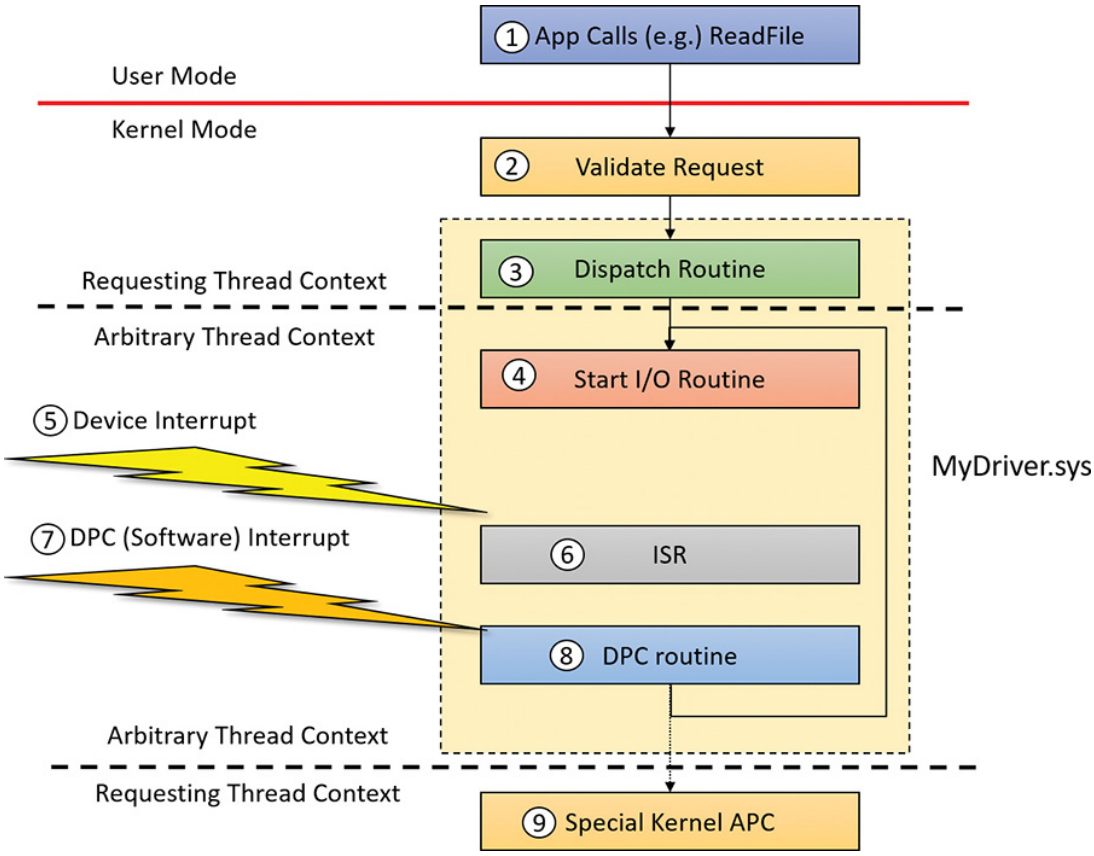


FIGURE 6-14 Typical single layer I/O request processing for hardware drivers.

Before we dig into the various steps outlined in **Figure 6-14**, some general comments are in order:

■ There are two types of horizontal divider lines. The first (solid line) is the usual user-mode/kernel-mode divider. The second (dotted line) separates code that runs in the requesting thread context versus the arbitrary thread context. These contexts are defined as follows:

• The requesting thread context region indicates that the executing thread is the original one that requested the I/O operation. This is important because if the thread is the one that made the original call, it means the process context is the original process, and so the user-mode address space that contains the user's buffer supplied to the I/O operation is directly accessible.

• The arbitrary thread context region indicates that the thread running those functions can be any thread. More specifically, it's most likely *not* the requesting thread, and so the user-mode process address space visi-

ble is not likely to be the original one. In this context, accessing the user's buffer with a user-mode address can be disastrous. You'll see in the next section how this issue is handled.

NOTE

The explanations for the steps outlined in **Figure 6-14** will prove why the divider lines reside where they are.

■ The large rectangle consisting of the four blocks (labeled Dispatch Routine, Start I/O Routine, ISR, and DPC Routine) represents the driver-provided code. All other blocks are provided by the system.

■ The figure assumes the hardware device can handle one operation at a time, which is true of many types of devices. Even if the device can handle multiple requests, the basic flow of operations is still the same.

Here is the sequence of events as outlined in **Figure 6-14**:

1. A client application calls a Windows API such as `ReadFile`. `ReadFile` calls the native `NtReadFile` (in Ntdll.dll), which makes the thread transition to kernel mode to the executive `NtReadFile` (these steps have already been discussed earlier in this chapter).

2. The I/O manager, in its `NtReadFile` implementation, performs some sanity checks on the request, such as whether the buffer provided by the client is accessible with the right page protection. Next, the I/O manager locates the associated driver (using the file handle provided), allocates and initializes an IRP, and calls the driver into the appropriate dispatch routine (in this case, corresponding to the `IRP_MJ_READ` index) using `IoCallDriver` with the IRP.

3. This is the first time the driver sees the IRP. This call is usually invoked using the requesting thread; the only way for that *not* to happen is if an upper filter held on to the IRP and called `IoCallDriver` later from a different thread. For the sake of this discussion, we'll assume

this is not the case (and in most cases involving hardware devices, this does not happen; even if there are upper filters, they do some processing and call the lower driver immediately from the same thread). The dispatch read callback in the driver has two tasks on its hand: first, it should perform more checking that the I/O manager can't do because it has no idea what the request really means. For example, the driver could check if the buffer provided to a read or write operation is large enough; or for a `DeviceIoControl` operation, the driver would check whether the I/O control code provided is a supported one. If any such check fails, the driver completes the IRP (`IoCompleteRequest`) with the failed status and returns immediately. If the checks turn up OK, the driver calls its Start I/O routine to initiate the operation. However, if the hardware device is currently busy (handling a previous IRP), then the IRP should be inserted into a queue managed by the driver and a `STATUS_PENDING` is returned without completing the IRP. The I/O manager caters for such a scenario with the `IoStartPacket` function, that checks a busy bit in the device object and, if the device is busy, inserts the IRP into a queue (also part of the device object structure). If the device is not busy, it sets the device bit as busy and calls the registered Start I/O routine (recall that there is such a member in the driver object that would have been initialized in `DriverEntry`). Even if a driver chooses not to use `IoStartPacket` , it would still follow similar logic.

4. If the device is not busy, the Start I/O routine is called from the dispatch routine directly—meaning it's still the requesting thread that is making the call. **Figure 6-14**, however, shows that the Start I/O routine is called in an arbitrary thread context; this will be proven to be true in the general case when we look at the DPC routine in step 8. The purpose of the Start I/O routine is to take the IRP relevant parameters and use them to program the hardware device (for example, by writing to its ports or registers using HAL hardware access routines such as `WRITE_PORT_UCHAR, WRITE_REGISTER_ULONG` , etc.). After the Start I/O completes, the call returns, and no particular code is running in the driver, the hardware is working and "does its thing." While the hardware device is working, more requests can come in to the device by the same thread (if using asynchronous operations) or other threads that

also opened handles to the device. In this case the dispatch routine would realize the device is busy and insert the IRP into the IRP queue (as mentioned, one way to achieve this is with a call to `IoStartPacket`).

5. When the device is done with the current operation, it raises an interrupt. The kernel trap handler saves the CPU context for whatever thread was running on the CPU that was selected to handle the interrupt, raises the IRQL of that CPU to the IRQL associated with the interrupt (DIRQL) and jumps to the registered ISR for the device.

6. The ISR, running at Device IRQL (above 2) does as little work as possible, telling the device to stop the interrupt signal and getting the status or other required information from the hardware device. As its last act, the ISR queues a DPC for further processing at a lower IRQL. The advantage of using a DPC to perform most of the device servicing is that any blocked interrupt whose IRQL lies between the Device IRQL and the DPC/dispatch IRQL (2) is allowed to occur before the lower-priority DPC processing occurs. Intermediate-level interrupts are thus serviced more promptly than they otherwise would be, and this reduces latency on the system.

7. After the interrupt is dismissed, the kernel notices that the DPC queue is not empty and so uses a software interrupt at IRQL DPC_LEVEL (2) to jump to the DPC processing loop.

8. Eventually, the DPC is de-queued and executes at IRQL 2, typically performing two main operations:

• It gets the next IRP in the queue (if any) and starts the new operation for the device. This is done first to prevent the device from being idle for too long. If the dispatch routine used `IoStartPacket`, then the DPC routine would call its counterpart, `IoStartNextPacket`, which does just that. If an IRP is available, the Start I/O routine is called from the DPC. This is why in the general case, the Start I/O routine is called in an arbitrary thread context. If there are no IRPs in the queue, the device is marked not busy—that is, ready for the next request that comes in.

• It completes the IRP, whose operation has just finished by the driver by calling `IoComplete-Request`. From that point, the driver is no longer responsible for the IRP and it shouldn't be touched, as it can be freed at any moment after the call. `IoCompleteRequest` calls any completion routines that have been registered. Finally, the I/O manager frees the IRP (it's actually using a completion routine of its own to do that).

9. The original requesting thread needs to be notified of the completion. Because the current thread executing the DPC is arbitrary, it's not the original thread with its original process address space. To execute code in the context of the requesting thread, a special kernel APC is issued to the thread. An APC is a function that is forced to execute in the context of a particular thread. When the requesting thread gets CPU time, the special kernel APC executes first (at IRQL `APC_LEVEL=1`). It does what's needed, such as releasing the thread from waiting, signaling an event that was registered in an asynchronous operation, and so on. (For more on APCs, see Chapter 8 in Part 2.)

A final note about I/O completion: the asynchronous I/O functions `ReadFileEx` and `WriteFileEx` allow a caller to supply a callback function as a parameter. If the caller does so, the I/O manager queues a user mode APC to the caller's thread APC queue as the last step of I/O completion. This feature allows a caller to specify a subroutine to be called when an I/O request is completed or canceled. User-mode APC completion routines execute in the context of the requesting thread and are delivered only when the thread enters an alertable wait state (by calling functions such as `SleepEx`, `WaitForSingleObjectEx`, or `WaitForMultipleObjectsEx`).

User address space buffer access

As shown in **Figure 6-14**, there are four main driver functions involved in processing an IRP. Some or all of these routines may need to access the buffer in user space provided by the client application. When an application or a device driver indirectly creates an IRP by using the `NtReadFile`, `NtWriteFile`, or `NtDeviceIoControlFile` system services (or the Windows API functions corresponding to these services, which are `ReadFile`, `WriteFile`, and `DeviceIoControl`), the pointer to the user's buffer is provided in the `UserBuffer` member of the IRP body. However, accessing this buffer directly can be done only in the requesting thread context (the client's process address space is visible) and in IRQL 0 (paging can be handled normally).

As discussed in the previous section, only the dispatch routine meets the criteria of running in the requesting thread context and in IRQL 0. And even this is not always the case—it's possible for an upper filter to hold on to the IRP and not pass it down immediately, possibly passing it down later on using a different thread, and could even be done when the CPU IRQL is 2 or higher.

The other three functions (Start I/O, ISR, DPC) clearly run on an arbitrary thread (could be any thread), and with IRQL 2 (DIRQL for the ISR). Accessing the user's buffer directly from any of these routine is mostly fatal. Here's why:

- Because the IRQL is 2 or higher, paging is not allowed. Since the user's buffer (or part of it) may be paged out, accessing the non-resident memory would crash the system.

- Because the thread executing these functions could be any thread, and thus a random process address space would be visible, the original user's address has no meaning and would likely lead to an access violation, or worse—accessing data from some random process (the parent process of whatever thread was running at the time).

Clearly, there must be a safe way to access the user's buffer in any of these routines. The I/O manager provides two options, for which it does the heavy lifting. These are known as Buffered I/O and Direct I/O. A third option, which is not really an option, is called Neither I/O, in which the I/O manager does nothing special and lets the driver handle the problem on its own.

A driver selects the method in the following way:

■ For read and write requests (`IRP_MJ_READ` and `IRP_MJ_WRITE`), it sets the `Flags` member (with an OR boolean operation so as not to disturb other flags) of the device object (`DEVICE_OBJECT`) to `DO_BUFFERED_IO` (for buffered I/O) or `DO_DIRECT_IO` (for direct I/O). If neither flag is set, neither I/O is implied. (`DO` is short for *device object*.)

■ For device I/O control requests (`IRP_MJ_DEVICE_CONTROL`), each control code is constructed using the `CTL_CODE` macro, where some of the bits indicate the buffering method. This means the buffering method can be set on a control code–by–control code basis, which is very useful.

The following sections describe each buffering method in detail.

Buffered I/O With buffered I/O, the I/O manager allocates a mirror buffer that is the same size as the user's buffer in non-paged pool and stores the pointer to the new buffer in the `AssociatedIrp.SystemBuffer` member of the IRP body. **Figure 6-15** shows the main stages in buffered I/O for a read operation (write is similar).

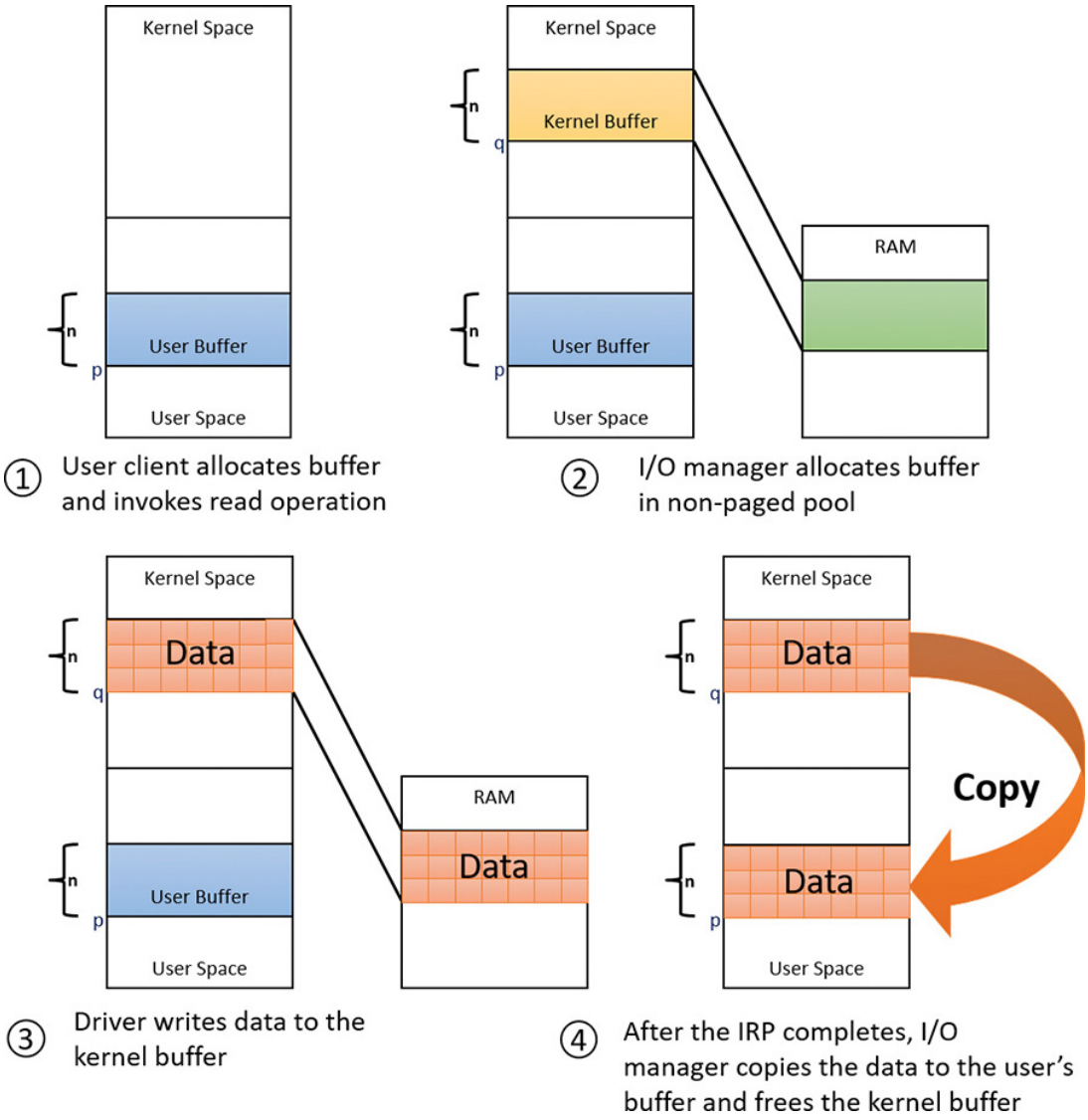


FIGURE 6-15 Buffered I/O.

The driver can access the system buffer (address *q* in **Figure 6-15**) from any thread and any IRQL:

■ The address is in system space, meaning it's valid in any process context.

■ The buffer is allocated from non-paged pool, so a page fault will not happen.

For write operations, the I/O manager copies the caller's buffer data into the allocated buffer when creating the IRP. For read operations, the I/O manager copies data from the allocated buffer to the user's buffer when the IRP completes (using a special kernel APC) and then frees the allocated buffer.

Buffered I/O clearly is very simple to use because the I/O manager does practically everything. Its main downside is that it always requires copying, which is inefficient for large buffers. Buffered I/O is commonly used when the buffer size is no larger than one page (4 KB) and when the device does not support direct memory access (DMA), because DMA is used to transfer data from a device to RAM or vice versa without CPU intervention—but with buffered I/O, there is always copying done with the CPU, which makes DMA pointless.

Direct I/O Direct I/O provides a way for a driver to access the user's buffer directly without any need for copying. **Figure 6-16** shows the main stages in direct I/O for a read or write operation.

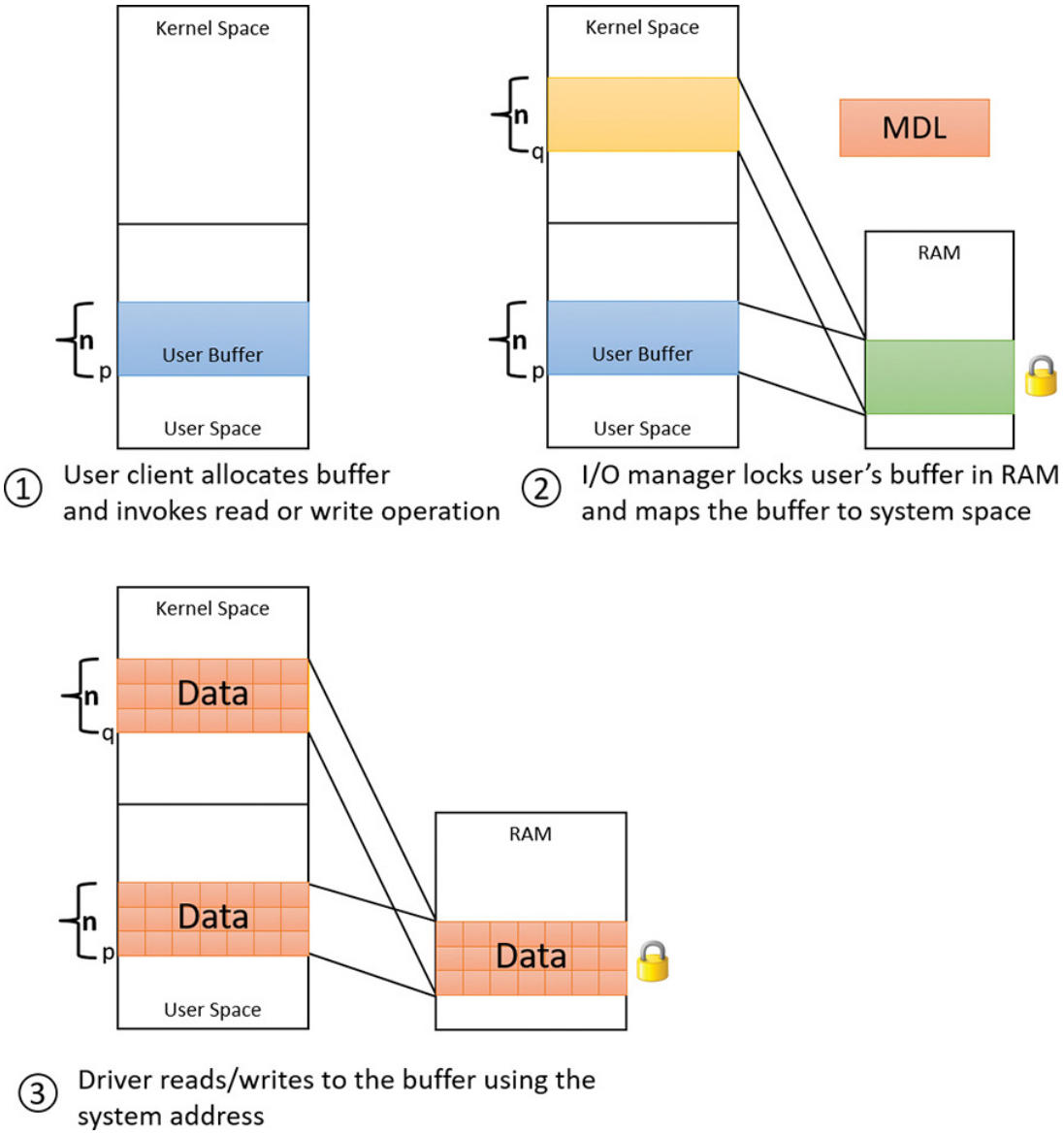


FIGURE 6-16 Direct I/O.

When the I/O manager creates the IRP, it locks the user's buffer into memory (that is, makes it non-pageable) by calling the `MmProbeAndLockPages` function (documented in the WDK). The I/O manager stores a description of the memory in the form of a memory descriptor list (MDL), which is a structure that describes the physical memory occupied by a buffer. Its address is stored in the `MdlAddress` member of the IRP body. Devices that perform DMA require only physical descriptions of buffers, so an MDL is sufficient for the operation of such devices. If a driver must access the contents of a buffer, however, it can map the buffer into the system's address space using the `MmGetSystemAddressForMdlSafe` function, passing in the provided MDL. The resulting pointer (*q* in **Figure 6-16**) is safe to use in any thread context (it's a system address) and in any IRQL (the buffer cannot be paged out). The user's buffer is effectively double-mapped, where the user's direct address (*p* in **Figure 6-16**) is usable only from the original process context, but the second mapping into system space is usable in any context. Once the IRP is complete, the I/O manager unlocks the buffer (making it pageable again) by calling `MmUnlockPages` (documented in the WDK).

Direct I/O is useful for large buffers (more than one page) because no copying is done, especially for DMA transfers (for the same reason).

Neither I/O With neither I/O, the I/O manager doesn't perform any buffer management. Instead, buffer management is left to the discretion of the device driver, which can choose to manually perform the steps the I/O manager performs with the other buffer-management types. In some cases, accessing the buffer in the dispatch routine is sufficient, so the driver may get away with neither I/O. The main advantage of neither I/O is its zero overhead.

Drivers that use neither I/O to access buffers that might be located in user space must take special care to ensure that buffer addresses are valid and do not reference kernel-mode memory. Scalar values, however, are perfectly safe to pass, although very few drivers have only a scalar value to pass around. Failure to do so could result in crashes or in security vulnerabilities, where applications have access to kernel-

mode memory or can inject code into the kernel. The `ProbeForRead` and `ProbeForWrite` functions that the kernel makes available to drivers verify that a buffer resides entirely in the user-mode portion of the address space. To avoid a crash from referencing an invalid user-mode address, drivers can access user-mode buffers protected with structured exception handling (SEH), expressed with `__try`/`__except` blocks in C/C++, that catch any invalid memory faults and translate them into error codes to return to the application. (See Chapter 8 in Part 2 for more information on SEH.) Additionally, drivers should also capture all input data into a kernel buffer instead of relying on user-mode addresses because the caller could always modify the data behind the driver's back, even if the memory address itself is still valid.

Synchronization

Drivers must synchronize their access to global driver data and hardware registers for two reasons:

- The execution of a driver can be preempted by higher-priority threads and time-slice (or quantum) expiration or can be interrupted by higher IRQL interrupts.

- On multiprocessor systems (the norm), Windows can run driver code simultaneously on more than one processor.

Without synchronization, corruption could occur—for example, device-driver code running at passive IRQL (0) (say, a dispatch routine) when a caller initiates an I/O operation can be interrupted by a device interrupt, causing the device driver's ISR to execute while its own device driver is already running. If the device driver was modifying data that its ISR also modifies—such as device registers, heap storage, or static data—the data can become corrupted when the ISR executes.

To avoid this situation, a device driver written for Windows must synchronize its access to any data that can be accessed at more than one IRQL. Before attempting to update shared data, the device driver must

lock out all other threads (or, in the case of a multiprocessor system, CPUs) to prevent them from updating the same data structure.

On a single-CPU system, synchronizing between two or more functions that run at different IRQLs is easy enough. Such function just needs to raise the IRQL (`KeRaiseIrql`) to the highest IRQL these functions execute in. For example, to synchronize between a dispatch routine (IRQL 0) and a DPC routine (IRQL 2), the dispatch routine needs to raise IRQL to 2 before accessing the shared data. If synchronization between a DPC and ISR is required, the DPC would raise IRQL to the Device IRQL (this information is provided to the driver when the PnP manager informs the driver of the hardware resources a device is connected to.) On multiprocessing systems, raising IRQL is not enough because the other routine—for example, ISR—could be serviced on another CPU (remember that IRQL is a CPU attribute, and not a global system attribute).

To allow high IRQL synchronization across CPUs, the kernel provides a specialized synchronization object: the spinlock. Here, we'll take a brief look at spinlocks as they apply to driver synchronization. (A full treatment of spinlocks is reserved for Chapter 8 in Part 2.) In principle, a spinlock resembles a mutex (also discussed in detail in Chapter 8 in Part 2) in the sense that it allows one piece of code to access shared data, but it works and is used quite differently. **Table 6-3** summarizes the differences between mutexes and spinlocks.

	Mutex	Spinlock
Synchronization nature	One thread out of any number of threads that is allowed enters a critical region and accesses shared data.	One CPU out of any number of CPUs that is allowed enters a critical region and accesses shared data.
Usable at IRQL	`< DISPATCH_LEVEL (2)`	`>= DISPATCH_LEVEL (2)`
Wait kind	Normal. That is, it does not waste CPU cycles while waiting.	Busy. That is, the CPU is constantly testing the spinlock bit until it's free.
Ownership	The owner thread is tracked, and recursive acquisition is allowed.	The CPU owner is not tracked, and recursive acquisition will cause a deadlock.

TABLE 6-3 Mutexes versus spinlocks

A spinlock is just a bit in memory that is accessed by an atomic test and modify operation. A spinlock may be owned by a CPU or free (unowned). As shown in **Table 6-3**, spinlocks are necessary when synchronization is needed in high IRQLs (>=2), because a mutex can't be used in

these cases as a scheduler is needed, but as we've seen the scheduler cannot wake up on a CPU whose IRQL is 2 or higher. This is why waiting for a spinlock is a busy wait operation: The thread cannot go to a normal wait state because that implies the scheduler waking up and switching to another thread on that CPU.

Acquiring a spinlock by a CPU is always a two-step operation. First, the IRQL is raised to the associated IRQL on which synchronization is to occur—that is, the highest IRQL on which the function that needs to synchronize executes. For example, synchronizing between a dispatch routine (IRQL 0) and a DPC (2) would need to raise IRQL to 2; synchronizing between DPC (2) and ISR (DIRQL) would need to raise IRQL to DIRQL (the IRQL for that particular interrupt). Second, the spinlock is attempted acquisition by atomically testing and setting the spinlock bit.

NOTE

The steps outlined for spinlock acquisition are simplified and omit some details that are not important for this discussion. The complete spinlock story is described in Chapter 8 in Part 2.

The functions that acquire spinlocks determine the IRQL on which to synchronize, as we shall see in a moment.

Figure 6-17 shows a simplified view of the two-step process of acquiring a spinlock.

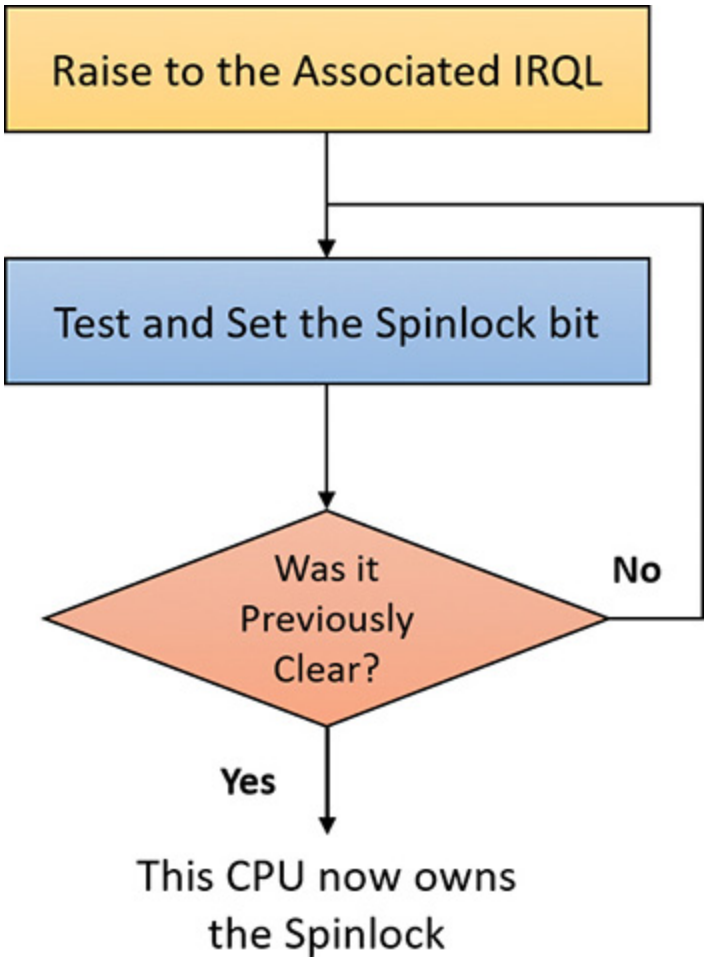


FIGURE 6-17 Spinlock acquisition.

When synchronizing at IRQL 2—for example, between a dispatch routine and a DPC or between a DPC and another DPC (running on another CPU, of course)—the kernel provides the `KeAcquireSpinLock` and `KeReleaseSpinLock` functions (there are other variations that are discussed in Chapter 8 in Part 2). These functions perform the steps in **Figure 6-17** where the "associated IRQL" is 2. The driver in this case must allocate a spinlock (`KSPIN_LOCK`, which is just 4 bytes on 32-bit systems and 8 bytes on 64-bit systems), typically in the device extension (where driver-managed data for the device is kept) and initialize it with `KeInitializeSpinLock`.

For synchronizing between any function (such as DPC or a dispatch routine) and the ISR, different functions must be used. Every interrupt object (`KINTERRUPT`) holds inside it a spinlock, which is acquired before the ISR executes (this implies that the same ISR cannot run concurrently on other CPUs). Synchronization in this case would be with that particular spinlock (no need to allocate another one), which can be ac-

quired indirectly with the `KeAcquireInterruptSpinLock` function and released with `KeReleaseInterruptSpinLock`. Another option is to use the `KeSynchronizeExecution` function, which accepts a callback function the driver provides that is called between the acquisition and release of the interrupt spinlock.

By now, you should realize that although ISRs require special attention, any data that a device driver uses is subject to being accessed by the same device driver (one of its functions) running on another processor. Therefore, it's critical for device-driver code to synchronize its use of any global or shared data or any accesses to the physical device itself.

I/O requests to layered drivers

The "**IRP flow**" section showed the general options drivers have for dealing with IRPs, with a focus on a standard WDM device node. The preceding section showed how an I/O request to a simple device controlled by a single device driver is handled. I/O processing for file-based devices or for requests to other layered drivers happens in much the same way, but it's worthwhile to take a closer look at a request targeted at file-system drivers. **Figure 6-18** shows a very simplified illustrative example of how an asynchronous I/O request might travel through layered drivers for non–hardware based devices as primary targets. It uses as an example a disk controlled by a file system.

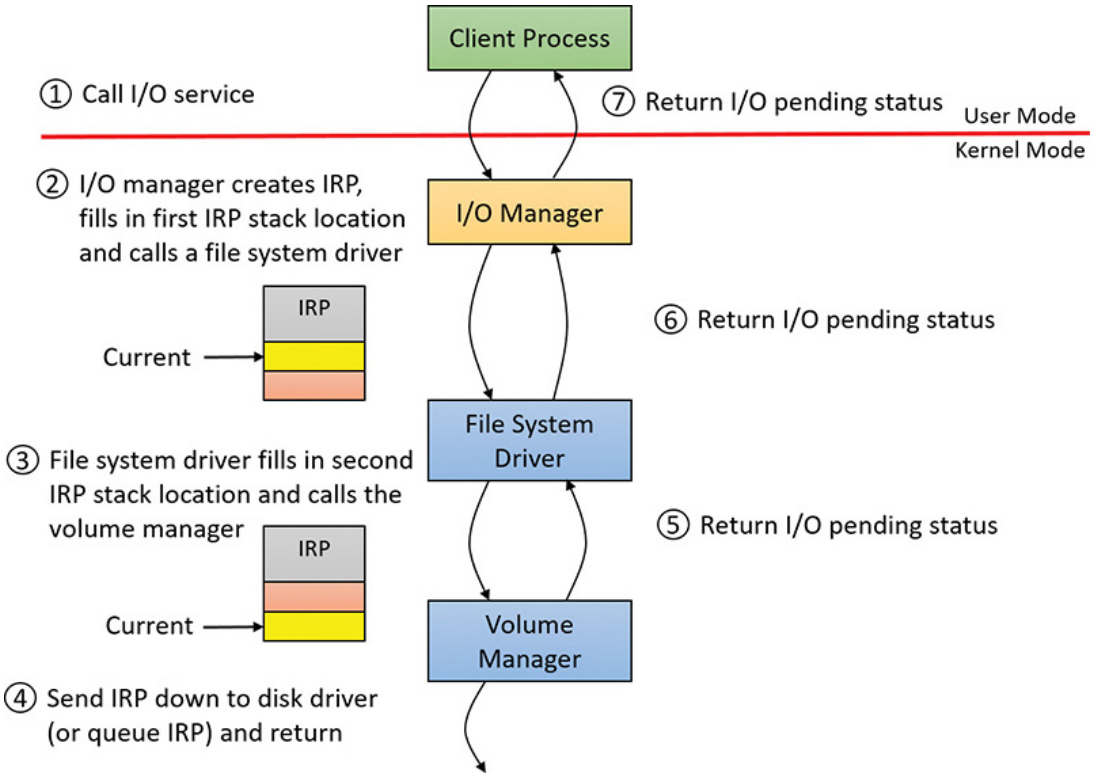


FIGURE 6-18 Queuing an asynchronous request to layered drivers.

Once again, the I/O manager receives the request and creates an IRP to represent it. This time, however, it delivers the packet to a file-system driver. The file-system driver exercises great control over the I/O operation at that point. Depending on the type of request the caller made, the file system can send the same IRP to the disk driver or it can generate additional IRPs and send them separately to the disk driver.

The file system is most likely to reuse an IRP if the request it receives translates into a single straightforward request to a device. For example, if an application issues a read request for the first 512 bytes in a file stored on a volume, the NTFS file system would simply call the volume manager driver, asking it to read one sector from the volume, beginning at the file's starting location.

After the disk controller's DMA adapter finishes a data transfer, the disk controller interrupts the host, causing the ISR for the disk controller to run, which requests a DPC callback completing the IRP, as shown in Figure 6-19.

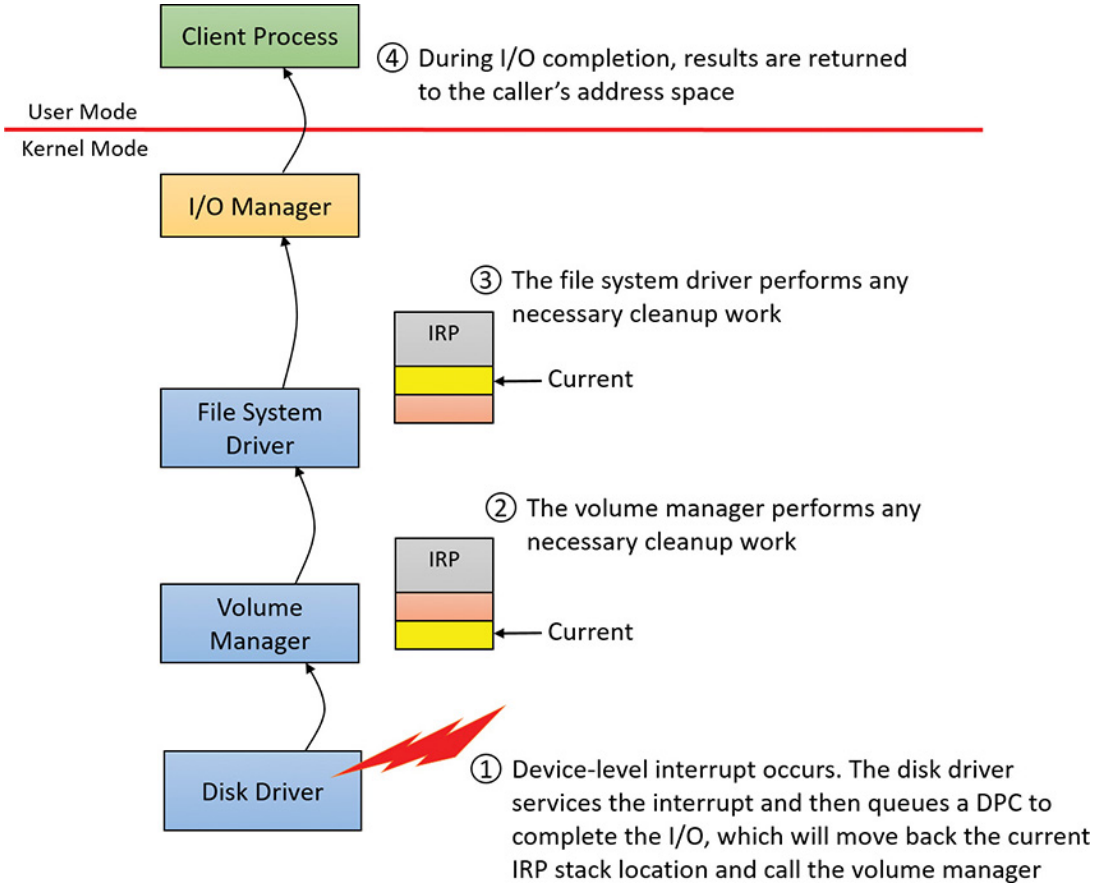


FIGURE 6-19 Completing a layered I/O request.

As an alternative to reusing a single IRP, a file system can establish a group of associated IRPs that work in parallel on a single I/O request. For example, if the data to be read from a file is dispersed across the disk, the file-system driver might create several IRPs, each of which reads some portion of the request from a different sector. This queuing is illustrated in **Figure 6-20**.

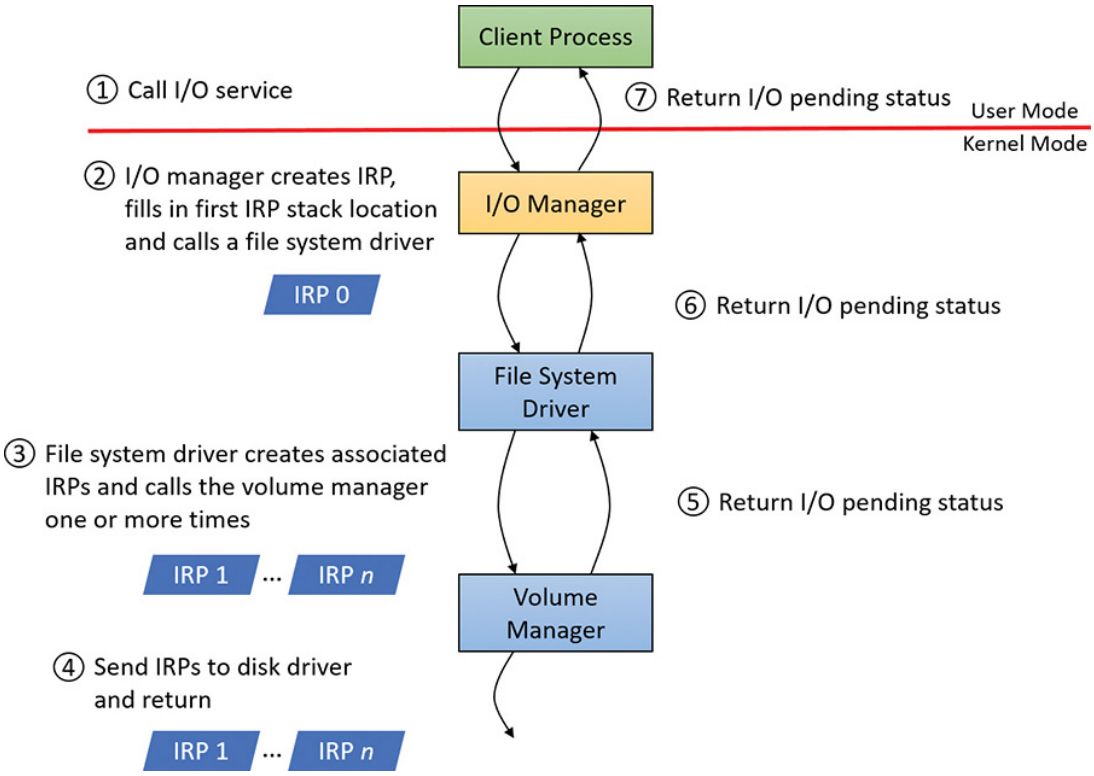


FIGURE 6-20 Queuing associated IRPs.

The file-system driver delivers the associated IRPs to the volume manager, which in turn sends them to the disk-device driver, which queues them to the disk device. They are processed one at a time, and the file-system driver keeps track of the returned data. When all the associated IRPs complete, the I/O system completes the original IRP and returns to the caller, as shown in **Figure 6-21**.

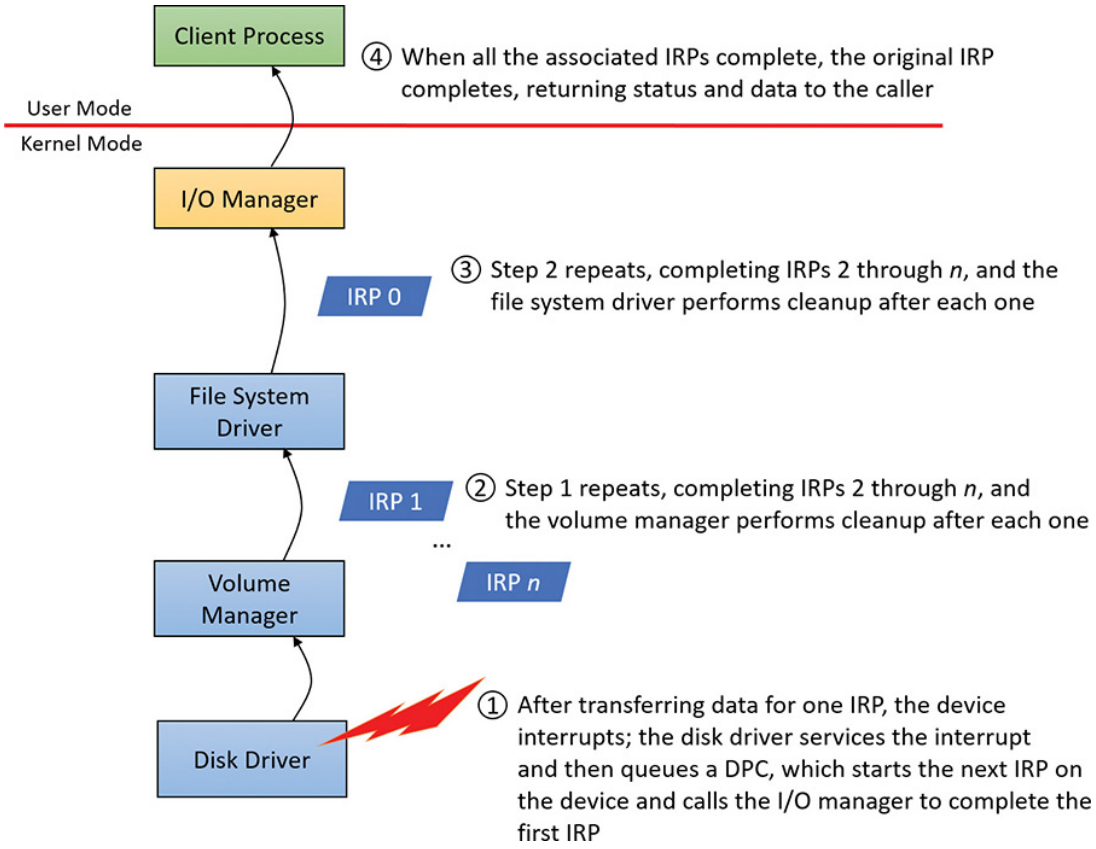


FIGURE 6-21 Completing associated IRPs.

NOTE

All Windows file-system drivers that manage disk-based file systems are part of a stack of drivers that is at least three layers deep. The file-system driver sits at the top, a volume manager in the middle, and a disk driver at the bottom. In addition, any number of filter drivers can be interspersed above and below these drivers. For clarity, the preceding example of layered I/O requests includes only a file-system driver and the volume-manager driver. See Chapter 12 in Part 2 for more information.

Thread-agnostic I/O

In the I/O models described thus far, IRPs are queued to the thread that initiated the I/O and are completed by the I/O manager issuing an APC to that thread so that process-specific and thread-specific context are accessible by completion processing. Thread-specific I/O processing is usually sufficient for the performance and scalability needs of most applications, but Windows also includes support for *thread-agnostic I/O* via two mechanisms:

- I/O completion ports, which are described at length in the section "**I/O completion ports**" later in this chapter

- Locking the user buffer into memory and mapping it into the system address space

With I/O completion ports, the application decides when it wants to check for the completion of I/O. Therefore, the thread that happens to have issued an I/O request is not necessarily relevant because any other thread can perform the completion request. As such, instead of completing the IRP inside the specific thread's context, it can be completed in the context of any thread that has access to the completion port.

Likewise, with a locked and kernel-mapped version of the user buffer, there's no need to be in the same memory address space as the issuing thread because the kernel can access the memory from arbitrary contexts. Applications can enable this mechanism by using `SetFileIoOverlappedRange` as long as they have the `SeLockMemoryPrivilege`.

With both completion port I/O and I/O on file buffers set by `SetFileIoOverlappedRange`, the I/O manager associates the IRPs with the file object to which they have been issued instead of with the issuing thread. The `!fileobj` extension in WinDbg shows an IRP list for file objects that are used with these mechanisms.

In the next sections, you'll see how thread-agnostic I/O increases the reliability and performance of applications in Windows.

I/O cancellation

While there are many ways in which IRP processing occurs and various methods to complete an I/O request, a great many I/O processing operations actually end in cancellation rather than completion. For example, a device may require removal while IRPs are still active, or the user might cancel a long-running operation to a device—for example, a network operation. Another situation that requires I/O cancellation support is thread and process termination. When a thread exits, the I/Os associated with the thread must be cancelled. This is because the I/O operations are no longer relevant and the thread cannot be deleted until the outstanding I/Os have completed.

The Windows I/O manager, working with drivers, must deal with these requests efficiently and reliably to provide a smooth user experience. Drivers manage this need by registering a cancel routine, by calling `IoSetCancelRoutine`, for their cancellable I/O operations (typically, those operations that are still enqueued and not yet in progress), which is invoked by the I/O manager to cancel an I/O operation. When drivers fail to play their role in these scenarios, users may experience unkill-

able processes, which have disappeared visually but linger and still appear in Task Manager or Process Explorer.

User-initiated I/O cancellation

Most software uses one thread to handle user interface (UI) input and one or more threads to perform work, including I/O. In some cases, when a user wants to abort an operation that was initiated in the UI, an application might need to cancel outstanding I/O operations. Operations that complete quickly might not require cancellation, but for operations that take arbitrary amounts of time—like large data transfers or network operations—Windows provides support for cancelling both synchronous and asynchronous operations.

- **Cancelling synchronous I/Os** A thread can call `CancelSynchronousIo`. This enables even create (open) operations to be cancelled when supported by a device driver. Several drivers in Windows support this functionality. These include drivers that manage network file systems (for example, MUP, DFS, and SMB), which can cancel open operations to network paths.

- **Cancelling asynchronous I/Os** A thread can cancel its own outstanding asynchronous I/Os by calling `CancelIo`. It can cancel all asynchronous I/Os issued to a specific file handle, regardless of which thread initiated them, in the same process with `CancelIoEx`. `CancelIoEx` also works on operations associated with I/O completion ports through the aforementioned thread-agnostic support in Windows. This is because the I/O system keeps track of a completion port's outstanding I/Os by linking them with the completion port.

Figure 6-22 and Figure 6-23 show synchronous and asynchronous I/O cancellation. (To a driver, all cancel processing looks the same.)

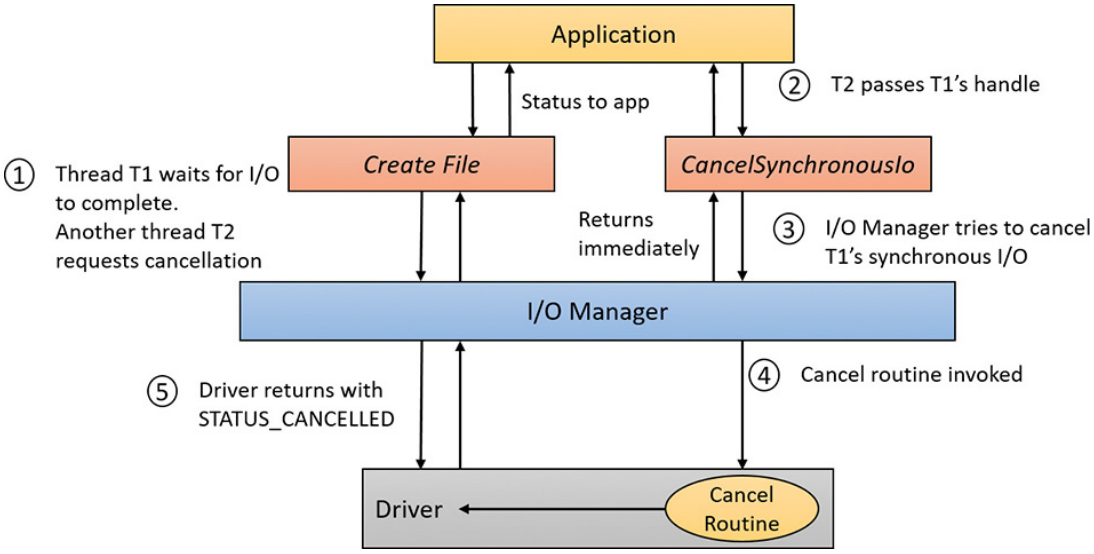


FIGURE 6-22 Synchronous I/O cancellation.

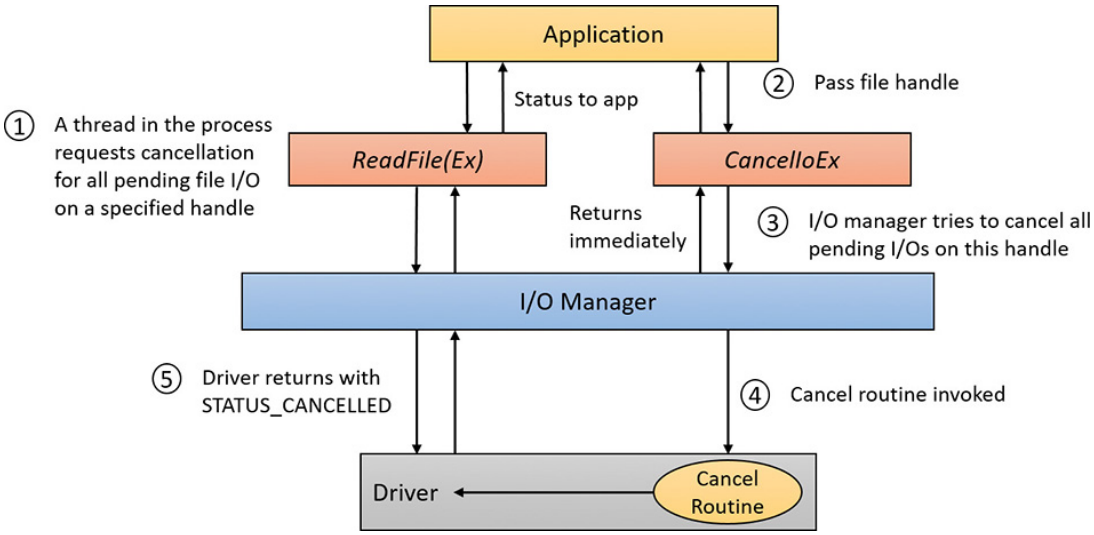


FIGURE 6-23 Asynchronous I/O cancellation.